

TRƯỜNG ĐẠI HỌC KỸ THUẬT - CÔNG NGHỆ CẦN THƠ (CTUET)
KHOA CÔNG NGHỆ THÔNG TIN**BÁO CÁO HỒ SƠ CHUYÊN MÔN TOÀN DIỆN TỪ A ĐẾN Z****CHƯƠNG 4: PHƯƠNG PHÁP TÍNH TOÁN BẤT ĐỒNG BỘ TRONG PYTHON****WORKSTREAM WS-01: NỀN TẢNG KHÁI NIỆM, BẢN ĐỒ TƯ DUY & PHÂN TÍCH HIỆU NĂNG**

Thuộc tính Hồ sơ	Thông tin Chi tiết Dự án
Học phần	Tính toán song song và phân tán (Parallel and Distributed Computing - PDC)
Giảng viên phụ trách	ThS. Lê Anh Nhã Uyên (Email: lanuyen@ctu.edu.vn)
Nhóm thực hiện	Nhóm 6 Thành viên (Dự án Nghiên cứu & Giảng giải Chuyên sâu Chương 4)
Thành viên phụ trách chính (TV1)	Thành viên 1 — Chủ trì WS-01 (Nền tảng, Bản đồ tư duy, Glossary & Data Contract)
Thành viên Reviewer / Backup (TV4)	Thành viên 4 — Chủ trì WS-04 (Event loop, Coroutine và Task)
Mốc kỹ thuật tham chiếu	Python 3.14.7 (Đối chiếu tài liệu chuẩn docs.python.org/3.14)
Định mức khối lượng TV1	100 điểm tải công việc chuẩn, 48 giờ lao động kỹ thuật, 8 Work Packages
Phạm vi slide sở hữu	8 slide chuẩn (1–8), ảnh xạ 6 slide rút gọn (R01–R06), 3 slide phụ lục (P01–P03)
Sản phẩm thực nghiệm	Data contract trạm cảm biến, Deterministic generator, Sequential Oracle Baseline

TUYÊN BỐ SỰ MỆNH VÀ NGUYÊN TẮC KỸ THUẬT CỦA TV1: Hồ sơ này đóng vai trò là 'Single Source of Truth' (Nguồn chân lý duy nhất) cho toàn bộ khối lượng công việc của Thành viên 1. Tài liệu giải đáp tường tận hai câu hỏi cốt lõi: WHAT (Bản chất khái niệm là gì ở mọi tầng kiến trúc) và HOW (Cơ chế hoạt động bên dưới, cách tổ chức mã nguồn, đo đặc hiệu năng và xử lý lỗi). Mọi phát biểu đều có căn cứ học thuật và đối chiếu tài liệu chính thức.

1. Tổng quan Trách nhiệm & Ma trận Truy vết của TV1

Trong cấu trúc nhóm 6 người, TV1 giữ vai trò 'người mở đường kiến thức' (Foundation & Concept Lead). Nếu TV1 giải thích sai hoặc mơ hồ về các khái niệm nền tảng (như đồng thời vs song song, blocking vs non-blocking), toàn bộ các phần tiếp theo của TV2 (ThreadPool), TV3 (ProcessPool), TV4 (Event loop), TV5 (Reliability) và TV6 (Hybrid Architecture) sẽ bị sụp đổ về mặt logic. Đồng thời, TV1 còn đảm nhiệm vai trò kiểm soát thuật ngữ (Glossary Lead) và thiết kế chuẩn dữ liệu đầu vào/đầu ra (Data Contract) cho bài toán demo chung.

1.1. Bảng phân rã 8 Gói công việc chuẩn (WBS: M1-01 đến M1-08)

Mã WP	Tên Gói công việc	Sản phẩm bàn giao & Tiêu chí nghiệm thu	Điểm	Giờ
M1-01	Khóa Taxonomy & Prerequisite	Bản Glossary chuẩn Anh-Việt, ma trận điều kiện tiên quyết, truy vết SRC-01..08	8	4h
M1-02	Hồ sơ Chuyên sâu Nền tảng	Báo cáo 4.500-5.500 từ, giải đáp WHAT/HOW, tối thiểu 5 nguồn học thuật/chuẩn CPython	20	10h
M1-03	Bộ Tài sản Học tập Trực quan	Ma trận thuật ngữ, timeline so sánh 4 task, sơ đồ process/thread/coroutine, 2 ví dụ đúng, 2 phản ví dụ	12	6h
M1-04	Thiết kế Slide & Speaker Notes	8 slide chuẩn, 6 slide rút gọn, 3 phụ lục P01-P03; speaker notes 1/3/8 phút; khớp thời lượng 8-9 phút	15	7h
M1-05	Đặc tả Data Contract & Baseline	Cấu trúc SensorReading, deterministic generator seed=42, sequential oracle, digest SHA-256, 6 test	15	7h
M1-06	Ngân hàng Câu hỏi & Teach-back	12 câu trắc nghiệm MCQ, 6 câu tự luận có rubric, 2 bài debug kèm code; dạy chéo 30 phút	10	5h
M1-07	Backup TV4 & Review Chéo	Biên bản review 5 tiêu chí khối TV4; thuyết trình thay phần TV4 (slide 25-32) trong diễn tập	10	5h
M1-08	Duy trì Glossary & GitHub Workflow	1 issue, 1 branch, >=3 commit chuẩn, 1 PR, review 2 PR khác; quản lý workload log	10	4h
TỔNG	HOÀN THÀNH TOÀN DIỆN TV1	Đáp ứng 100% tiêu chí nghiệm thu Gate G1 đến Gate G7 của dự án	100	48h

1.2. Ma trận Truy vết Nguồn bài giảng gốc (Traceability Matrix)

Tất cả các nội dung do TV1 phụ trách đều xuất phát từ tài liệu bài giảng gốc của môn học (file BaiGiang_TinhToanSongSong_PhanTan_C4.pdf), được chuẩn hóa mã hóa truy vết từ SRC-01 đến SRC-08 và mở rộng bằng các yêu cầu học thuật hiện đại EXT-01, EXT-02:

Mã Yêu cầu	Nội dung gốc trong Slide Bài giảng C4	Slide sở hữu	Mục tiêu mở rộng kỹ thuật của TV1
SRC-01	Trang 1: Bìa bài giảng Chương 4 - Tính toán bất đồng bộ	Slide 1	Đặt bài toán trung tâm: Hệ thống giám sát trạm cảm biến môi trường; nêu 3 câu hỏi cốt lõi
SRC-02	Trang 2: Xử lý tuần tự (Sequential) - 1 task/thời gian, theo thứ tự	Slide 1–2	Vẽ timeline tuần tự chính xác; chỉ rõ thời gian CPU idle do nghẽn I/O wait; xây sequential baseline
SRC-03	Trang 3: Concurrency - Tiến trình, CPU core chia nhỏ task, xen kẽ	Slide 2–3	Phân tích time-slicing, luồng điều khiển, cơ chế lập lịch preemptive vs cooperative của OS/runtime
SRC-04	Trang 4: Concurrency - Điều phối nhiều task, tại 1 thời điểm chỉ 1 task	Slide 3–4	Làm rõ bản chất tiến triển (progress) trong khoảng thời gian; phân biệt rõ với parallelism
SRC-05	Trang 5: Parallelism - Nhiều task trong cùng 1 thời điểm, độc lập, core > 1	Slide 2, 4	Phân tích điều kiện đa nhân vật lý, vectorization, độc lập dữ liệu, phân biệt thread vs process
SRC-06	Trang 6: Parallelism - Thực tế trên mỗi nhân vẫn xử lý đồng thời	Slide 4, 7	Làm rõ hardware threads vs software threads, hiện tượng oversubscription và contention
SRC-07	Trang 7: Asynchronous - Không theo thứ tự, giải quyết nhiều yêu cầu	Slide 3, 5	Xây dựng ma trận 2x2 phân biệt rõ rệt Asynchronous vs Non-blocking; tránh nhầm lẫn tai hại
SRC-08	Trang 8: Mô hình bất đồng bộ thực thi đồng thời - Luồng đơn duy nhất	Slide 3, 8	Giải thích cơ chế single-thread event loop, điểm yield/suspend, chuẩn bị bàn giao cho TV4
EXT-01	Mở rộng 1: Phân loại Workload I/O-bound vs CPU-bound	Slide 5	Dấu hiệu nhận diện, công cụ profiling (perf_counter, cProfile), mini-decision tree chọn mô hình
EXT-02	Mở rộng 2: Nền tảng Đo lường, Định luật Amdahl & Overhead	Slide 6–7	Chứng minh toán học Amdahl, đồ thị bão hòa speedup, 4 overhead thực tế, nguyên tắc Correctness First

2. Giải mã Bản đồ Tư duy: 7 Khái niệm Cốt lõi (WHAT & HOW)

Một trong những nguyên nhân lớn nhất khiến sinh viên và kỹ sư thất bại khi tiếp cận lập trình bất đồng bộ là 'đánh đồng thuật ngữ'. Các từ khóa như Concurrency, Parallelism, Asynchrony, Non-blocking thường bị dùng lẫn lộn. Phần này mổ xẻ tận gốc 7 khái niệm theo cả hai chiều: Bản chất là gì (WHAT) và Hoạt động như thế nào bên dưới hệ thống (HOW).

2.1. Xử lý Tuần tự (Sequential Computing)

WHAT (Bản chất): Mô hình lập trình cơ bản và trực giác nhất, trong đó các lệnh và tác vụ được thực thi nối tiếp nhau theo một thứ tự xác định.

- Tại mỗi thời điểm, hệ thống chỉ thực thi đúng một chỉ thị duy nhất của một tác vụ duy nhất.
- Tác vụ B chỉ có thể bắt đầu khi và chỉ khi tác vụ A đã hoàn tất 100% chu kỳ sống của nó.
- Tổng thời gian thực thi của N tác vụ bằng tổng thời gian của từng tác vụ cộng lại: $T_{total} = \sum(T_i)$.

HOW (Cơ chế hoạt động bên dưới): Về mặt phần cứng và hệ điều hành, CPU duy trì một con trỏ lệnh Program Counter (PC) trỏ tới địa chỉ lệnh tiếp theo trong bộ nhớ, kết hợp cùng một Ngăn xếp thực thi (Call Stack) đơn. Khi một hàm được gọi, một Stack Frame được đẩy vào ngăn xếp. Điểm yếu chí mạng của tuần tự xảy ra khi tác vụ thực hiện thao tác I/O blocking (như đọc dữ liệu từ cảm biến qua mạng hoặc đọc đĩa): CPU phải dừng hoàn toàn (stall) hoặc chuyển trạng thái luồng sang BLOCKED/SLEEPING. Suốt khoảng thời gian trễ mạng (network latency), hàng tỷ chu kỳ xung nhịp của CPU bị lãng phí hoàn toàn vào việc chờ đợi nhân rồi (idle wait).

2.2. Xử lý Đồng thời (Concurrency)

WHAT (Bản chất): Khả năng cấu trúc, quản lý và điều phối nhiều tác vụ khác nhau cùng có tiến triển (making progress) trong cùng một khoảng thời gian.

- Concurrency là một thuộc tính về mặt cấu trúc (structure / composition) của chương trình, không phải thuộc tính về phần cứng.
- Rob Pike (tác giả ngôn ngữ Go) định nghĩa kinh điển: 'Concurrency is about dealing with lots of things at once. Parallelism is about doing lots of things at once.' (Đồng thời là xử lý/sắp đặt nhiều việc cùng lúc; Song song là thực sự chạy nhiều việc cùng lúc).
- Hai tác vụ A và B được gọi là đồng thời nếu khoảng thời gian thực thi $[start_A, end_A]$ và $[start_B, end_B]$ có phần giao nhau.

HOW (Cơ chế hoạt động bên dưới): Trên một hệ thống đơn nhân (single-core CPU), đồng thời đạt được thông qua cơ chế Phân chia thời gian (Time-slicing) và Xen kẽ (Interleaving). Hệ thống điều phối chia nhỏ công việc của các tác vụ thành các lát thời gian (time slices / quanta). Khi Tác vụ A rơi vào trạng thái chờ I/O hoặc hết lát thời gian, Bộ lập lịch (Scheduler) sẽ thực hiện Chuyển ngữ cảnh (Context Switch) để nạp Tác vụ B vào thực thi. Nhờ tốc độ chuyển đổi cực nhanh (hàng nghìn lần mỗi giây), người dùng có cảm giác máy tính đang làm nhiều việc cùng lúc, nhưng tại một thời điểm vật lý cực vi mô (tại một pico-giây), CPU core chỉ đang thi hành chỉ thị của đúng một tác vụ.

2.3. Xử lý Song song (Parallelism)

WHAT (Bản chất): Kỹ thuật thực thi đồng thời về mặt vật lý (physically simultaneous execution) của nhiều tác vụ tại cùng một thời điểm.

- Parallelism là một thuộc tính về mặt thực thi (execution / hardware property).

- Điều kiện tiên quyết bắt buộc: Hệ thống phần cứng phải có số đơn vị xử lý độc lập lớn hơn 1 (số nhân vật lý CPU core > 1, đa CPU socket, hoặc hàng nghìn lõi GPU).
- Mục tiêu tối thượng: Rút ngắn thời gian tính toán của các bài toán nặng về tính toán (CPU-bound) bằng cách chia nhỏ dữ liệu hoặc bài toán.

HOW (Cơ chế hoạt động bên dưới): Các tác vụ hoàn toàn độc lập được nạp vào các thanh ghi và pipeline của các nhân CPU riêng biệt. Tại cùng một thời điểm xung nhịp t_0 , Nhân 1 (Core 1) đang thực thi chỉ thị của Tác vụ A, trong khi Nhân 2 (Core 2) đang thực thi chỉ thị của Tác vụ B. Không có sự nhường nhịn hay chia sẻ thời gian giữa A và B trên cùng một nhân. Tuy nhiên, trên thực tế, trên mỗi nhân CPU độc lập đó, Hệ điều hành vẫn áp dụng cơ chế xử lý đồng thời (interleaving) để phục vụ hàng trăm tiến trình nền khác nhau của hệ thống.

Minh họa trực quan: Sơ đồ so sánh trực thời gian thực thi của 4 tác vụ A, B, C, D qua 3 mô hình:

 *Đoạn mã minh họa: Trục thời gian (Timeline) so sánh Tuần tự vs Đồng thời vs Song song*

TUẦN TỰ (Sequential - 1 Core):

Core 0: |--Task A--|-----Task B-----|--Task C--|----Task D----|---> Thời gian (t)
Tổng thời gian = $T(A) + T(B) + T(C) + T(D)$. CPU lãng phí khi tác vụ chờ I/O.

ĐỒNG THỜI (Concurrent / Interleaving - 1 Core):

Core 0: |-A1-|-B1-|-A2-|...I/O wait...|-C1-|-B2-|-D1-|-C2-|-D2-|---> Thời gian (t)
Chỉ 1 Core duy nhất, các task xen kẽ nhau. Tận dụng thời gian rảnh khi task khác chờ I/O.

SONG SONG (Parallel - 2 Cores):

Core 0: |-----Task A-----|-----Task C-----|-----> Thời gian (t)
Core 1: |-----Task B-----|-----Task D-----|-----> Thời gian (t)
Hai tác vụ chạy cùng lúc tại cùng thời điểm vật lý trên hai phần cứng riêng biệt.

2.4. Đồng bộ (Synchronous) vs Bất đồng bộ (Asynchronous)

WHAT (Bản chất): Khái niệm mô tả Hợp đồng giao tiếp (Communication Contract) giữa Bên gọi (Caller) và Bên được gọi (Callee) về thời điểm và cách thức bàn giao kết quả.

- Synchronous (Đồng bộ): Caller gọi hàm và bị giữ lại, chỉ tiếp tục khi Callee hoàn thành và trả kết quả về trong cùng một ngữ cảnh điều khiển.
- Asynchronous (Bất đồng bộ): Caller khởi tạo yêu cầu và lập tức nhận lại quyền điều khiển (cùng một đối tượng đại diện như Future, Task, Promise) mà không cần đợi kết quả hoàn tất.

HOW (Cơ chế hoạt động bên dưới): Trong mô hình bất đồng bộ, Callee thực hiện công việc ở chế độ nền (background) hoặc đăng ký sự kiện với hệ thống I/O. Caller có thể tiếp tục làm các công việc tính toán khác. Khi Callee có kết quả hoặc gặp lỗi, hệ thống sẽ thông báo cho Caller thông qua một trong ba cơ chế: (1) Callback function được đăng ký trước; (2) Kiểm tra trạng thái định kỳ (Polling trên Future); hoặc (3) Điểm tạm dừng và tiếp tục hợp tác (biểu thức await trong Coroutine).

2.5. Blocking vs Non-blocking

WHAT (Bản chất): Khái niệm mô tả Trạng thái của Luồng thực thi (Execution Thread) tại tầng Lờ gọi Hệ thống (System Call) và Giao tiếp Phần cứng/Hệ điều hành.

- Blocking: Thao tác làm dừng (treo) luồng hiện tại cho đến khi sự kiện I/O sẵn sàng hoặc dữ liệu được nạp đủ vào bộ đệm.
- Non-blocking: Thao tác lập tức trả quyền điều khiển về cho luồng gọi ngay cả khi dữ liệu chưa sẵn sàng, đi kèm mã trạng thái báo hiệu (như EAGAIN, EWOULDBLOCK).

HOW (Cơ chế hoạt động bên dưới): Khi thực hiện lời gọi blocking (ví dụ: `socket.recv()`), luồng chuyển từ trạng thái RUNNING sang BLOCKED trong bảng quản lý tiến trình của OS kernel. Kernel loại luồng này khỏi Ready Queue của CPU cho đến khi card mạng kích hoạt ngắt phần cứng (Hardware Interrupt) báo dữ liệu đã về RAM. Ngược lại, với socket non-blocking, kernel kiểm tra bộ đệm socket; nếu trống, kernel lập tức trả về lỗi EWOULDBLOCK mà không bắt luồng phải ngủ. Để không phải chạy vòng lặp bận kiểm tra liên tục (busy-wait), hệ thống sử dụng các cơ chế I/O Multiplexing cấp kernel (như epoll trên Linux, kqueue trên macOS, hoặc I/O Completion Ports - IOCP trên Windows) để giám sát đồng thời hàng chục nghìn socket và chỉ đánh thức luồng khi có sự kiện thực sự phát sinh.

2.6. Ma trận 2x2: Đồng bộ / Bất đồng bộ kết hợp Blocking / Non-blocking

Đây là điểm gây nhầm lẫn phổ biến nhất: nhiều người lầm tưởng Synchronous đồng nghĩa với Blocking, và Asynchronous đồng nghĩa với Non-blocking. Trên thực tế, đây là hai trục độc lập hoàn toàn. Bảng ma trận 2x2 dưới đây giải mã 4 sự kết hợp trong thực tế kỹ thuật:

Trục Kết hợp	Đặc điểm Kỹ thuật	Tác động Runtime	Ví dụ Minh họa trong Python
1. Synchronous + Blocking (Truyền thống)	Caller chờ callee xong mới đi tiếp; trong lúc chờ, luồng bị hệ điều hành đưa vào trạng thái ngủ (BLOCKED).	Đơn giản, trực quan, an toàn logic; nhưng lãng phí tài nguyên CPU nghiêm trọng khi có I/O trễ mạng.	<pre>import requests resp = requests.get('http://station1/data') # Luồng dừng cứng chờ server trả lời.</pre>
2. Synchronous + Non-blocking (Busy-waiting / Polling)	Caller vẫn chờ kết quả của callee; nhưng callee trả về ngay; caller phải chủ động lặp lại kiểm tra liên tục.	Tạo tải ảo, làm CPU utilization nhảy vọt lên 100% (đốt cháy chu kỳ CPU vô ích chỉ để hỏi 'đã xong chưa?').	<pre>sock.setblocking(False) while True: try: data = sock.recv(1024); break except BlockingIOError: pass</pre>
3. Asynchronous + Blocking (Anti-pattern tai hại)	Giao diện gọi là bất đồng bộ (trả về Task/Future); nhưng bên trong tác vụ lại thực thi mã blocking đồng bộ.	LỖI CHÍ MẠNG: Làm đóng băng toàn bộ Event Loop thread, khiến tất cả các coroutine khác bị treo dây chuyền.	<pre>async def read_station(): time.sleep(2) # BLOCK TOÀN LOOP! return {'status': 'ok'}</pre>
4. Asynchronous + Non-blocking (Mô hình Lý tưởng)	Caller nhận handle ngay; I/O bên dưới là non-blocking đăng ký với event loop qua epoll/IOCP; thread tự do phục vụ việc khác.	Hiệu quả tài nguyên tối đa; một luồng đơn lẻ có thể phục vụ hàng chục nghìn kết nối I/O đồng thời.	<pre>async def read_station(): await asyncio.sleep(2) # #Nhường loop return {'status': 'ok'}</pre>

❗ QUY TẮC VÀNG VỀ 'AWAIT' VÀ BLOCKING TRONG ASYNCIO: Từ khóa 'await' trong Python KHÔNG LÀM HÀM CHẠY NHANH HƠN và KHÔNG LÀM MÃ BLOCKING BIẾN THÀNH NON-BLOCKING. Biểu thức 'await' thực chất là một điểm tạm dừng (yield point). Nó chỉ có ý nghĩa khi đối tượng được await là một awaitable non-blocking thực sự (như `asyncio.sleep`, `asyncio.open_connection`). Nếu bạn đặt `'time.sleep()'` hoặc `'requests.get()'` vào trong một hàm `async def`, bạn đang tạo ra cấu hình Asynchronous + Blocking (Ồ số 3) và làm tê liệt toàn bộ ứng dụng!

3. Ba Đơn vị Thực thi & Cơ chế Lập lịch (Process vs Thread vs Coroutine)

Để hiểu sâu về tính toán đồng thời và bất đồng bộ, lập trình viên bắt buộc phải nắm vững bản chất kiến trúc của ba đơn vị thực thi cơ bản: Tiến trình (Process), Luồng (Thread) và Hàm đồng quy (Coroutine). Mỗi đơn vị có mô hình bộ nhớ, chi phí khởi tạo, chi phí chuyển ngữ cảnh và cơ chế lập lịch hoàn toàn khác biệt.

3.1. Tiến trình (Process)

WHAT (Bản chất): Một chương trình đang trong quá trình thực thi, được Hệ điều hành cấp phát một Không gian địa chỉ ảo (Virtual Address Space) hoàn toàn độc lập, bảng mô tả tệp (File Descriptor Table), và định danh tiến trình duy nhất (PID).

- Cô lập bộ nhớ tuyệt đối: Tiến trình A không thể tự ý đọc hay ghi vào vùng nhớ của Tiến trình B nhờ sự bảo vệ của phần cứng MMU (Memory Management Unit).
- Tính chịu lỗi cao: Một tiến trình bị crash (Segmentation Fault, Out Of Memory) hoàn toàn không ảnh hưởng đến sự sống còn của các tiến trình khác.
- Chi phí nặng nề: Cấp phát trang bộ nhớ (Page Tables), nạp mã nhị phân, khởi tạo Python Interpreter riêng biệt.

HOW (Cơ chế hoạt động bên dưới): Trên Linux, tiến trình được tạo qua cơ chế 'fork' (sao chép bằng trang bộ nhớ với cơ chế Copy-on-Write) hoặc 'spawn' (khởi tạo mới tinh). Trên Windows, Python chỉ hỗ trợ cơ chế 'spawn' (phải khởi động lại trình thông dịch Python từ đầu, nạp lại toàn bộ module, đây là lý do bắt buộc phải có câu lệnh bảo vệ 'if __name__ == "__main__":' để tránh đệ quy vô hạn). Do không chia sẻ bộ nhớ, khi hai tiến trình muốn trao đổi dữ liệu, chúng phải thông qua cơ chế Truyền thông Liên tiến trình (IPC - Inter-Process Communication) như Pipes, Unix Sockets, hoặc Shared Memory. Trong Python, mọi đối tượng gửi qua IPC bắt buộc phải được tuần tự hóa (Serialization) bằng thư viện 'pickle', đây là nút thắt cổ chai cực lớn về CPU và độ trễ nếu truyền dữ liệu kích thước lớn.

3.2. Luồng (Thread)

WHAT (Bản chất): Đơn vị lập lịch CPU nhẹ nhất (Lightweight Process) nằm bên trong một tiến trình. Tất cả các luồng trong cùng một tiến trình chia sẻ chung toàn bộ không gian địa chỉ: Heap, Mã lệnh (Code segment), Biến toàn cục (Data segment), và Tệp mở.

- Mỗi luồng chỉ sở hữu riêng: Ngăn xếp thực thi (Stack - lưu biến cục bộ và call trace) và Tập thanh ghi CPU (Registers bao gồm Program Counter).
- Truyền thông tức thời: Các luồng trao đổi dữ liệu thông qua bộ nhớ chia sẻ mà không cần tuần tự hóa (no pickling overhead).
- Hiểm họa đồng bộ: Rủi ro tranh chấp tài nguyên (Race Condition) và Khóa chết (Deadlock) khi nhiều luồng cùng ghi vào một vùng nhớ chung.

HOW (Cơ chế hoạt động bên dưới & Tác động của GIL): Hệ điều hành quản lý luồng bằng cơ chế Lập lịch Tranh đoạt (Preemptive Scheduling). Kernel OS sử dụng bộ đếm thời gian phần cứng (Hardware Timer Interrupt) để ngắt luồng đang chạy sau mỗi lát thời gian (10-20ms) và cưỡng bức chuyển CPU sang luồng khác. Chi phí chuyển ngữ cảnh luồng (Thread Context Switch) tốn khoảng 1-2 microsecond (do phải lưu/khôi phục thanh ghi CPU, đổi con trỏ ngăn xếp và có thể gây cache miss).

ĐẶC BIỆT TRONG CPYTHON — KHÓA THÔNG DỊCH TOÀN CỤC (GIL - Global Interpreter Lock): CPython sử dụng cơ chế đếm tham chiếu (Reference Counting) để quản lý bộ nhớ. Để tránh việc hai

luồng cùng tăng/giảm biến đếm tham chiếu gây hỏng bộ nhớ, CPython đặt một khóa đơn duy nhất tên là GIL bao bọc toàn bộ máy ảo. Hệ quả kỹ thuật: Tại bất kỳ thời điểm nào, CHỈ CÓ ĐÚNG MỘT LUỒNG ĐƯỢC PHÉP THỰC THI BYTECODE PYTHON, ngay cả khi máy tính có 64 nhân CPU vật lý! Do đó, đa luồng trong Python KHÔNG THỂ TĂNG TỐC các tác vụ tính toán thuần túy (CPU-bound). Luồng trong Python chỉ hữu hiệu cho các tác vụ chờ mạng/đĩa (I/O-bound) vì khi thực hiện system call I/O hoặc các thư viện C tối ưu (như NumPy, OpenCV), CPython sẽ chủ động giải phóng GIL (release GIL) cho luồng khác chạy.

3.3. Hàm Đồng quy (Coroutine)

WHAT (Bản chất): Đơn vị thực thi logic siêu nhẹ ở không gian người dùng (User-space), được điều khiển hoàn toàn bởi Lập trình viên và Khung điều phối (Event Loop), không có sự can thiệp của Nhân Hệ điều hành (Kernel).

- Coroutine là một hàm có khả năng tạm dừng (suspend) việc thực thi tại các điểm xác định và sau đó có thể được tiếp tục (resume) lại chính xác từ trạng thái đó.
- Trong Python, coroutine function được định nghĩa bằng cú pháp 'async def'. Khi được gọi, nó không chạy ngay mà trả về một 'coroutine object'.
- Bộ nhớ cực nhỏ: Một coroutine object chỉ tốn khoảng 1-2 KB RAM (chỉ là một đối tượng Frame trên heap của Python), so với 1-8 MB bộ đệm ngăn xếp mặc định của một OS Thread.

HOW (Cơ chế hoạt động bên dưới): Coroutine hoạt động theo cơ chế Lập lịch Hợp tác (Cooperative Scheduling / Cooperative Multitasking). Hệ điều hành hoàn toàn KHÔNG BIẾT đến sự tồn tại của các coroutine; đối với OS, toàn bộ ứng dụng asyncio chỉ là MỘT LUỒNG DUY NHẤT (Single OS Thread). Trong luồng duy nhất đó, Vòng lặp Sự kiện (Event Loop) đóng vai trò là nhạc trưởng điều phối:

1. Event loop lấy một tác vụ (Task) từ Ready Queue ra chạy.
2. Tác vụ đó chạy liên tục cho đến khi gặp một biểu thức 'await <awaitable>'.
3. Nếu awaitable đó chưa hoàn thành (ví dụ: đang chờ socket đọc từ cảm biến), tác vụ sẽ TỰ NGUYỆN NHƯỜNG QUYỀN (yield) lại cho Event loop.
4. Event loop đăng ký socket đó vào bộ giám sát hệ thống (epoll/kqueue/IOCP), rồi lập tức chuyển CPU sang chạy một Task khác đang sẵn sàng trong Ready Queue.
5. Khi socket có dữ liệu, epoll đánh thức event loop; loop đưa Task ban đầu trở lại Ready Queue để chạy tiếp đoạn mã sau lệnh await.

Chi phí chuyển ngữ cảnh giữa các coroutine (User-space Context Switch) chỉ mất vài chục nanosecond (chỉ đơn giản là đổi con trỏ frame trong Python), nhanh hơn hàng nghìn lần so với chuyển ngữ cảnh luồng của hệ điều hành!

3.4. Bảng So sánh Toàn diện: Process vs Thread vs Coroutine

Tiêu chí So sánh	Tiến trình (Process)	Luồng (Thread)	Coroutine (Hàm đồng quy)
Tầng quản lý	Nhân hệ điều hành (OS Kernel)	Nhân hệ điều hành (OS Kernel)	Không gian người dùng (User-space / Event loop)
Cơ chế Lập lịch	Tranh đoạt (Preemptive)	Tranh đoạt (Preemptive)	Hợp tác tự nguyện (Cooperative multitasking)
Không gian bộ nhớ	Cô lập hoàn toàn (Private VAS)	Chia sẻ chung Heap/Data trong process	Chia sẻ chung bộ nhớ của thread chứa nó
Chi phí bộ nhớ (RAM)	Lớn (~15–30 MB/process Python)	Trung bình (8 MB stack ảo, ~100 KB thực)	Rất nhẹ (~1–2 KB/coroutine)

Chi phí Khởi tạo	Rất nặng (~10–50 ms)	Trung bình (~50–200 μ s)	Siêu nhẹ (< 1 μ s)
Chi phí Context Switch	Rất nặng (~5–15 μ s, flush TLB)	Trung bình (~1–3 μ s, lưu thanh ghi)	Cực nhẹ (~20–50 ns, đổi frame con trỏ)
Khả năng tận dụng đa nhân	Rất tốt (Mỗi process 1 core riêng biệt)	Bị chặn bởi GIL trên CPython	Chỉ chạy trên 1 core (Single-threaded loop)
Truyền thông nội bộ	IPC qua Pipes/Sockets (Pickle)	Bộ nhớ chia sẻ trực tiếp (No pickle)	Biến trực tiếp / asyncio.Queue
Hiểm họa lập trình	Deadlock IPC, cạn kiệt RAM	Race condition, Deadlock bộ nhớ	Treo loop nếu gọi blocking, task bị quên
Workload phù hợp nhất	CPU-bound nặng thuần Python	I/O-bound khi thư viện là blocking	I/O-bound mạng/socket quy mô lớn (C10K)

4. Phân loại Workload: I/O-bound, CPU-bound & Workload Hỗn hợp

Một trong những nguyên tắc cốt tử mà TV1 phải truyền tải là: 'Không có công cụ nào tối ưu cho mọi bài toán; việc lựa chọn mô hình đồng thời phụ thuộc hoàn toàn vào BẮN CHẮT NÚT THẮT CỔ CHAI (Bottleneck) của bài toán'. Trước khi viết bất kỳ dòng mã nào, kỹ sư phải đo lường và phân loại chính xác bài toán thuộc nhóm nào.

4.1. Workload Hạn chế bởi Nhập/Xuất (I/O-bound Workload)

WHAT (Bản chất): Bài toán mà tổng thời gian hoàn thành chủ yếu bị chi phối bởi thời gian chờ đợi các thiết bị ngoại vi hoặc tài nguyên bên ngoài: đọc/ghi tệp trên đĩa cứng, gửi/nhận gói tin qua mạng Internet, truy vấn cơ sở dữ liệu, hoặc đọc cảm biến từ xa.

- Triệu chứng đo lường: Mức sử dụng CPU (CPU Utilization) rất thấp (thường < 10-15%), trong khi thời gian thực tế trôi qua (Wall-clock time) lại rất lớn.
- Quan hệ thời gian: $T_{\text{waiting}} \gg T_{\text{compute}}$.
- Mục tiêu tối ưu: Không phải làm cho từng thao tác I/O chạy nhanh hơn (vì ta không thể ép mạng Internet nhanh hơn), mà là CHỒNG LẤP (OVERLAP) các khoảng thời gian chờ đợi nhân rồi.
- Công cụ ứng viên: 'asyncio' (nếu có thư viện async non-blocking chuẩn) hoặc 'ThreadPoolExecutor' (để bọc các API I/O blocking truyền thống).

4.2. Workload Hạn chế bởi Tính toán (CPU-bound Workload)

WHAT (Bản chất): Bài toán mà tổng thời gian hoàn thành bị chi phối hoàn toàn bởi năng lực xử lý số học và logic của chip CPU: tính toán ma trận, mã hóa/giải mã mật mã (AES, RSA), nén dữ liệu (gzip, zlib), xử lý ảnh, huấn luyện mô hình máy học.

- Triệu chứng đo lường: Mức sử dụng CPU đạt đỉnh 100% trên một hoặc nhiều nhân; thời gian CPU User Time xấp xỉ bằng Wall-clock time.
- Quan hệ thời gian: $T_{\text{compute}} \gg T_{\text{waiting}}$.
- Mục tiêu tối ưu: Tận dụng tối đa phần cứng đa nhân (Parallelism) để thực thi song song các phần tính toán độc lập.
- Công cụ ứng viên: 'ProcessPoolExecutor' (trên CPython có GIL để tách thành các tiến trình độc lập trên các nhân CPU khác nhau), hoặc dùng thư viện native giải phóng GIL (NumPy).

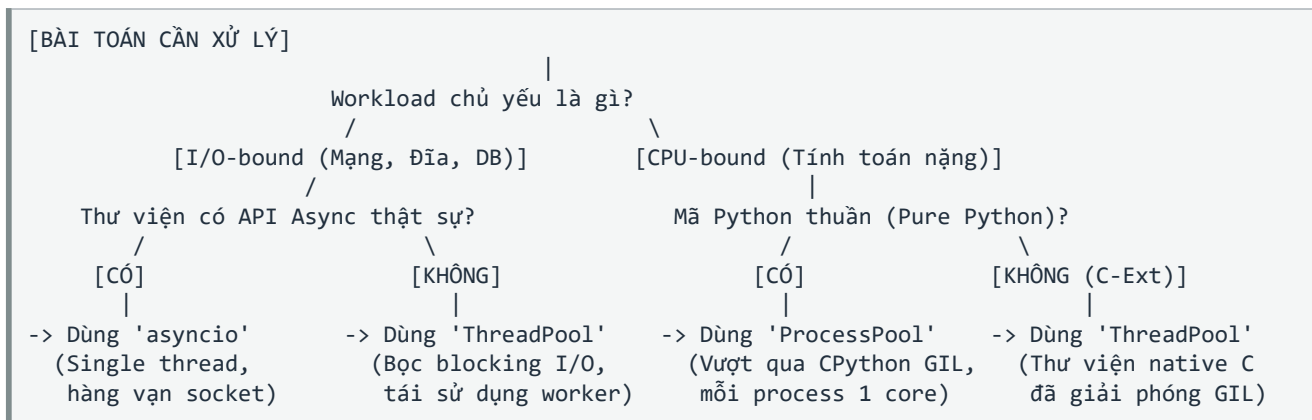
4.3. Workload Hỗn hợp (Mixed Workload)

Bản chất thực tế: Đại đa số các hệ thống công nghiệp trong thực tế là Workload hỗn hợp. Diễn hình như bài toán trung tâm của nhóm: Đọc dữ liệu từ trạm cảm biến môi trường qua mạng (I/O-bound), sau đó làm sạch và tính toán chỉ số chất lượng không khí tổng hợp bằng thuật toán thống kê (CPU-bound), cuối cùng ghi log và phát cảnh báo (I/O-bound).

HOW (Giải pháp kiến trúc): Chiến lược kiến trúc tối ưu: Áp dụng Mô hình Lai (Hybrid Architecture). Dùng 'asyncio' ở tầng ngoài cùng để điều phối hàng nghìn kết nối mạng với chi phí bộ nhớ tối thiểu; khi dữ liệu về, offload công đoạn tính toán CPU nặng sang một nhóm tiến trình công nhân (ProcessPool) thông qua hàm 'loop.run_in_executor()'; kết hợp cùng hàng đợi hữu hạn (asyncio.Queue có maxsize) để tạo áp lực ngược (Backpressure), ngăn không cho tầng nhận dữ liệu làm tràn ngập tầng tính toán.

4.4. Cây Quyết định Chọn Mô hình Thực thi (Decision Tree)

 Đoạn mã minh họa: Cây quyết định lựa chọn công cụ đồng thời trong Python



5. Nền tảng Đo lường Hiệu năng, Định luật Amdahl & Bốn Ngộ nhận

Một chương trình chạy nhanh nhưng trả về kết quả sai là hoàn toàn vô giá trị. Nguyên tắc cốt tử đầu tiên của kỹ thuật đo lường là: 'Correctness First, Performance Later' (Kiểm chứng Tính đúng đắn trước, Đo hiệu năng sau). Phần này thiết lập cơ sở toán học và phương pháp luận đo lường chuẩn xác cho cả nhóm.

5.1. Bốn Chỉ số Hiệu năng Cốt lõi

- **Độ trễ (Latency):** Thời gian cần thiết để hoàn thành một tác vụ đơn lẻ từ lúc gửi yêu cầu đến lúc nhận kết quả cuối cùng (đơn vị: mili-giây ms hoặc giây s).
- **Thông lượng (Throughput):** Số lượng tác vụ được hoàn tất trên một đơn vị thời gian (đơn vị: transactions per second - TPS, hoặc requests/giây). Một hệ thống có thể có throughput cao dù latency của từng request không giảm (nhờ xử lý đồng thời nhiều request).
- **Hệ số Tăng tốc (Speedup - S):** Tỷ số giữa thời gian chạy tuần tự và thời gian chạy song song: $S(N) = T_{seq} / T_{par}(N)$.
- **Hiệu suất Tài nguyên (Efficiency - E):** Tỷ số giữa Speedup và số lượng worker/nhân phần cứng: $E(N) = S(N) / N$. Trong điều kiện lý tưởng tuyệt đối, $E = 1.0$ (100%); trong thực tế, E luôn nhỏ hơn 1 và giảm dần khi N tăng.

5.2. Định luật Amdahl (Amdahl's Law) & Chứng minh Toán học

WHAT (Bản chất): Được Gene Amdahl công bố năm 1967, định luật này xác lập giới hạn lý thuyết tối đa về hệ số tăng tốc (Speedup) mà một hệ thống có thể đạt được khi nâng cấp tài nguyên song song, với điều kiện khối lượng công việc (Workload) là cố định.

HOW (Chứng minh công thức & Bài toán mẫu): Giả sử một chương trình gồm hai phần: Phần có thể song song hóa được chiếm tỷ lệ p ($0 \leq p \leq 1$), và phần bắt buộc phải chạy tuần tự chiếm tỷ lệ $s = (1 - p)$.

Nếu chạy trên 1 worker, tổng thời gian là $T(1) = s \cdot T(1) + p \cdot T(1)$.

Khi triển khai trên N worker chạy song song lý tưởng, phần p được chia đều cho N worker, trong khi phần s vẫn giữ nguyên:

$$T(N) = (1 - p) \cdot T(1) + (p / N) \cdot T(1)$$

Do đó, Hệ số Tăng tốc $S(N)$ được xác định bởi công thức:

$$S(N) = T(1) / T(N) = 1 / [(1 - p) + (p / N)]$$

GIỚI HẠN TIỆM CẬN (Asymptotic Limit):

Khi số lượng worker N tiến tới vô cùng ($N \rightarrow \infty$), đại lượng p/N tiến về 0. Giới hạn trên của Speedup sẽ là:

$$S_{\max} = \lim_{N \rightarrow \infty} S(N) = 1 / (1 - p)$$

BÀI TOÁN TÍNH TOÁN THỰC TẾ:

Giả sử trong bài toán xử lý trạm cảm biến của nhóm: Bước khởi tạo kết nối và tổng hợp báo cáo cuối cùng chiếm 20% thời gian ($s = 1 - p = 0.20$), còn bước đọc cảm biến và phân tích chiếm 80% ($p = 0.80$).

- Với $N = 2$ worker: $S(2) = 1 / [0.20 + 0.80/2] = 1 / 0.60 = 1.67$ lần.

- Với $N = 4$ worker: $S(4) = 1 / [0.20 + 0.80/4] = 1 / 0.40 = 2.50$ lần.

- Với $N = 8$ worker: $S(8) = 1 / [0.20 + 0.80/8] = 1 / 0.30 = 3.33$ lần.

- Với $N = 1000$ worker: $S(1000) = 1 / [0.20 + 0.80/1000] = 1 / 0.2008 = 4.98$ lần.

- Khi $N \rightarrow \infty$: Speedup TỐI ĐA KHÔNG BAO GIỜ VƯỢT QUÁ: $S_{\max} = 1 / 0.20 = 5.0$ LẦN!

Kết luận: Dù bạn có đầu tư siêu máy tính 1 triệu lõi CPU, chương trình cũng không thể nhanh hơn 5 lần!

⚠ CẢNH BÁO KỸ THUẬT VỀ CHI PHÍ OVERHEAD THỰC TẾ: ĐỊNH LUẬT AMDAHL VẪN CÒN QUÁ LẠC QUAN SO VỚI THỰC TẾ! Công thức Amdahl giả định chi phí quản lý song song bằng 0. Trong thực tế, hệ thống luôn phải gánh chịu 4 loại CHI PHÍ OVERHEAD:

- Chi phí khởi tạo worker: Spawn process, cấp phát thread stack.
- Chi phí IPC & Serialization: Pickle/unpickle dữ liệu truyền giữa các tiến trình.
- Chi phí tranh chấp khóa (Lock Contention): Các worker tranh chấp tài nguyên chung gây nghẽn hàng đợi.
- Nghẽn băng thông phần cứng: Memory bus saturation và cache invalidation.

Nếu thêm các chi phí này vào mô hình, đường cong Speedup sẽ đạt đỉnh ở một số lượng worker nhất định rồi SUY GIẢM (Nhiều worker hơn lại làm chương trình chạy CHẬM HƠN).

5.3. Bốn Ngộ nhận Phổ biến (Myths vs. Technical Facts)

Ngộ nhận Phổ biến (Myth)	Sự thật Kỹ thuật (Technical Fact)	Giải thích Cơ chế Bên dưới
Ngộ nhận 1: 'Chuyển sang lập trình bất đồng bộ (async) chắc chắn chương trình sẽ chạy nhanh hơn.'	SAI. Async không làm một phép tính chạy nhanh hơn; nó chỉ tăng thông lượng bằng cách tận dụng thời gian rảnh khi chờ I/O.	Nếu bài toán là CPU-bound hoặc chỉ có 1 task đơn lẻ, overhead của Event loop và Task wrapping thậm chí làm chương trình chạy CHẬM HƠN tuần tự.
Ngộ nhận 2: 'Càng tăng nhiều worker'	SAI. Tăng worker vượt quá số nhân vật	Gây hiện tượng Oversubscription: Hệ

trong ThreadPool hoặc ProcessPool thì tốc độ càng tăng.'	lý hoặc ngưỡng nghẽn I/O sẽ gây suy thoái hiệu năng nghiêm trọng.	điều hành mất hết thời gian cho Context Switching (trashing), tranh chấp khóa bộ nhớ và bão hòa băng thông.
Ngộ nhận 3: 'Mô hình async chạy trên 1 luồng duy nhất (Single-thread Event Loop) nên không bao giờ bị Race Condition.'	SAI HOÀN TOÀN. Dù không có tranh đoạt cấp CPU, Race Condition vẫn xảy ra ở mức Ứng dụng tại các điểm await.	Nếu hai coroutine cùng đọc một biến toàn cục, sau đó cùng gọi 'await asyncio.sleep()', rồi cùng cập nhật biến đó, kết quả sẽ bị ghi đè (lost update).
Ngộ nhận 4: 'Lập trình bất đồng bộ trên một máy tính tương đương với Hệ thống Phân tán (Distributed Computing).'	SAI VỀ RANH GIỚI BẢN CHẤT. Hai mô hình chia sẻ tư duy event-driven nhưng khác biệt hoàn toàn về mô hình lỗi và vùng nhớ.	Trên một máy, bộ nhớ được bảo vệ bởi OS kernel, không có rủi ro phân mảnh mạng (network partition), không có độ trễ vật lý xa, và crash tiến trình là cục bộ.

5.4. Quy trình 10 Bước Đo lường Hiệu năng Công bằng (Fair Benchmark Protocol)

Để kết quả đo đạc giữa 5 chế độ thực thi (Sequential, ThreadPool, ProcessPool, Asyncio, Hybrid) của nhóm có giá trị khoa học, TV1 ban hành quy trình 10 bước chuẩn bắt buộc toàn nhóm phải tuân thủ nghiêm ngặt:

- Bước 1: Cố định Khối lượng Công việc (Fixed Workload):** Toàn bộ các chế độ phải chạy trên cùng một bộ dữ liệu đầu vào (cùng số lượng trạm, cùng seed ngẫu nhiên).
- Bước 2: Kiểm chứng Tính đúng đắn trước (Correctness Verification):** Kiểm tra mã băm Result Digest (SHA-256) của chế độ mới có khớp hoàn toàn với Sequential Baseline hay không. Nếu sai digest, loại bỏ kết quả benchmark ngay lập tức.
- Bước 3: Tách biệt Môi trường Đo (Isolation):** Tắt các ứng dụng chạy nền nặng, cố định cấu hình máy tính (ghi rõ CPU model, số core vật lý/logic, RAM, hệ điều hành).
- Bước 4: Sử dụng Đồng hồ Đo chuẩn xác:** Bắt buộc dùng 'time.perf_counter()' hoặc 'time.perf_counter_ns()' để đo thời gian trôi qua thực tế (wall-clock time), tuyệt đối không dùng 'time.time()' vì có thể bị méo do đồng bộ NTP.
- Bước 5: Chạy Khởi động (Warm-up Phase):** Thực hiện ít nhất 1 lần chạy khởi động trước khi bắt đầu đo để làm ấm CPU cache, nạp sẵn dynamic libraries và kích hoạt JIT/bytecode compilation.
- Bước 6: Lặp lại Nhiều lần (Multiple Iterations):** Đo tối thiểu 5 đến 10 lần chạy độc lập cho mỗi chế độ.
- Bước 7: Báo cáo bằng Trung vị (Median) và Phương sai:** Không chỉ đưa ra một con số tốt nhất; bắt buộc phải lấy giá trị Trung vị (Median) và khoảng biến thiên (min, max, standard deviation) để loại trừ đột biến do OS background tasks.
- Bước 8: Loại bỏ I/O Console trong Vùng đo:** Tuyệt đối không gọi 'print()' hoặc ghi log dày đặc bên trong đoạn mã đang bấm giờ vì tốc độ xuất console của terminal sẽ làm sai lệch hoàn toàn kết quả.
- Bước 9: Đo lường Đa chiều:** Không chỉ đo tổng thời gian (wall time), phải đo thêm CPU time (user/system time), bộ nhớ đỉnh (Peak Memory RSS via tracemalloc/psutil), và tỷ lệ lỗi.
- Bước 10: Minh bạch Hạn chế:** Công khai phiên bản Python (ví dụ: Python 3.14.7), tham số hệ điều hành và ghi rõ ranh giới: kết quả này chỉ đúng trên môi trường thử nghiệm cụ thể.

6. Đặc tả Chi tiết Module Demo của TV1 (Data Contract & Sequential Baseline)

Để đảm bảo 5 chế độ thực thi (Sequential, Thread, Process, Async, Hybrid) trong toàn dự án có thể so sánh công bằng và khoa học, TV1 chịu trách nhiệm thiết kế Hợp đồng Dữ liệu (Data Contract), Bộ sinh

dữ liệu giả lập xác định (Deterministic Generator), và Xây dựng Bản chuẩn Tuần tự (Sequential Baseline) làm Chuẩn mực Kiểm chứng Tính đúng đắn (Correctness Oracle). Mọi thành viên khác (TV2 đến TV6) bắt buộc phải tích hợp và đối soát kết quả với module này.

6.1. Hợp đồng Dữ liệu Bản ghi Cảm biến (Data Contract Schema)

Trường dữ liệu (Field)	Kiểu dữ liệu (Type)	Khoảng giá trị hợp lệ (Range)	Mô tả Kỹ thuật
station_id	str	STA-001 đến STA-999	Mã định danh duy nhất của trạm cảm biến môi trường
timestamp	float	Epoch timestamp (giây)	Thời điểm ghi nhận dữ liệu tại trạm
temperature	float	-10.0 đến 60.0 (°C)	Nhiệt độ môi trường đo được
humidity	float	0.0 đến 100.0 (%)	Độ ẩm tương đối của không khí
air_quality_index	float	0.0 đến 500.0 (AQI)	Chỉ số bụi mịn / ô nhiễm không khí
status	str	'OK', 'WARN', 'ERROR'	Trạng thái hoạt động của cảm biến tại trạm
network_delay	float	0.05 đến 0.50 (giây)	Độ trễ truyền thông mô phỏng của mạng không dây

6.2. Mã nguồn Hoàn chỉnh Module: sensor_baseline.py

 Đoạn mã minh họa: Mã nguồn chuẩn sensor_baseline.py của TV1

```
# sensor_baseline.py
# Module do TV1 phát triển: Data Contract & Sequential Correctness Oracle
import time
import random
import hashlib
from dataclasses import dataclass
from typing import List, Dict, Any, Tuple

@dataclass(frozen=True)
class SensorReading:
    station_id: str
    timestamp: float
    temperature: float
    humidity: float
    air_quality_index: float
    status: str
    network_delay: float

def generate_sensor_batch(n_stations: int = 8, seed: int = 42) -> List[SensorReading]:
    """Sinh bộ dữ liệu giả lập xác định bằng seed cố định để đảm bảo tính tái lập."""
    rng = random.Random(seed)
    readings = []
    base_time = 1775000000.0 # Epoch mốc chuẩn

    for i in range(1, n_stations + 1):
```

```

        station_id = f"STA-{i:03d}"
        if i == 4:
            status = "ERROR"
            aqi = -99.0 # Tiềm lỗi có chủ đích
        else:
            status = "OK"
            aqi = round(rng.uniform(20.0, 180.0), 2)

        reading = SensorReading(
            station_id=station_id,
            timestamp=base_time + i * 10,
            temperature=round(rng.uniform(22.0, 38.0), 2),
            humidity=round(rng.uniform(45.0, 90.0), 2),
            air_quality_index=aqi,
            status=status,
            network_delay=round(rng.uniform(0.08, 0.20), 3)
        )
        readings.append(reading)
    return readings

def compute_sensor_metric(reading: SensorReading) -> Dict[str, Any]:
    """Thao tác CPU-bound nhỏ: phân tích chỉ số bất thường và composite score."""
    if reading.status != "OK" or reading.air_quality_index < 0:
        return {
            "station_id": reading.station_id,
            "valid": False,
            "health_score": 0.0,
            "alert": "SENSOR_FAULT_DETECTED"
        }

    # Tính toán chỉ số sức khỏe tổng hợp (Composite Environmental Health Index)
    acc = 0.0
    for step in range(2000):
        acc += (reading.temperature * 0.4 + reading.humidity * 0.2 + reading.air_quality_index *
0.4) / (step + 1)

    health_score = round(acc % 100.0, 4)
    alert = "HEALTHY" if reading.air_quality_index <= 100 else "UNHEALTHY"
    return {
        "station_id": reading.station_id,
        "valid": True,
        "health_score": health_score,
        "alert": alert
    }

def process_sequential(readings: List[SensorReading]) -> Tuple[List[Dict[str, Any]], Dict[str,
Any]]:
    """Sequential Baseline Processor: Chuẩn mực kiểm chứng tính đúng đắn (Correctness
Oracle)."""
    t_start = time.perf_counter()
    results = []

    for reading in readings:
        # Bước 1: Mô phỏng I/O network delay (blocking sleep)
        time.sleep(reading.network_delay)

        # Bước 2: Tính toán chỉ số nghiệp vụ (CPU work)
        metric = compute_sensor_metric(reading)
        results.append(metric)

```

```
duration = time.perf_counter() - t_start

# Bước 3: Tính toán Result Digest (SHA-256) trên toàn bộ kết quả để đối soát
raw_str = "|".join([f"{r['station_id']}:{r['valid']}:{r['health_score']}:{r['alert']}" for r
in results])
result_digest = hashlib.sha256(raw_str.encode("utf-8")).hexdigest()

metadata = {
    "mode": "sequential",
    "station_count": len(readings),
    "duration_seconds": round(duration, 4),
    "result_digest": result_digest,
    "valid_count": sum(1 for r in results if r["valid"]),
    "error_count": sum(1 for r in results if not r["valid"])
}
return results, metadata
```

6.3. Bộ 6 Kiểm thử Bắt buộc (Verification Test Matrix)

Mã Test	Tên Kiểm thử	Mục tiêu & Kỳ vọng (Expected Result)	Phương thức Thực thi
TEST-01	Tính Xác định (Determinism)	Với cùng seed=42, hai lần gọi generate_sensor_batch phải trả về chính xác 100% dữ liệu từng trường	assert readings1 == readings2
TEST-02	Bảo toàn Schema (Contract)	Mọi bản ghi sinh ra phải tuân thủ đúng dataclass, không có trường None, đúng kiểu dữ liệu	assert isinstance(r, SensorReading)
TEST-03	Kiểm chứng Oracle (Correctness)	Bản chuẩn tuần tự phải tính toán đúng công thức composite metric cho tất cả trạm hợp lệ	assert r['health_score'] > 0
TEST-04	Phát hiện Lỗi (Fault Detection)	Trạm STA-004 bị tiêm lỗi phải được phân loại chính xác là valid=False và alert='SENSOR_FAULT_DETECTED'	assert results[3]['valid'] is False
TEST-05	Đo lường Baseline (Timing)	Thời gian tuần tự phải xấp xỉ tổng network_delay của các trạm: T_seq >= SUM(delay_i)	assert duration >= sum_delay
TEST-06	Bất biến Digest (Digest Invariance)	Mã băm SHA-256 của kết quả phải cố định tuyệt đối giữa các lần chạy tuần tự lặp lại	assert meta1['result_digest'] == meta2['result_digest']

7. Chuẩn 40 Năng lực Cốt lõi & Kiến thức cần nắm của TV1

Theo Quy ước Dự án (D02, D04), mỗi thành viên phải nắm vững toàn diện 40 năng lực chung của Chương 4, trong đó TV1 chịu trách nhiệm sở hữu chuyên sâu Cụm Nền tảng (Năng lực 1–10) và Cụm

Đo lường Hiệu năng (Năng lực 37–40), đồng thời làm Chuyên gia Dự phòng (Backup) cho toàn bộ Cụm Event Loop & Asyncio của TV4 (Năng lực 21–30).

7.1. Phân tích Chuyên sâu 10 Năng lực Nền tảng do TV1 Sở hữu Chính

STT	Năng lực Cốt lõi	Bản chất Kỹ thuật cần nắm vững của TV1
1	Bài toán tính toán bất đồng bộ giải quyết	Giải quyết bài toán lãng phí chu kỳ CPU khi chương trình phải chờ đợi các thao tác I/O chậm chạp (mạng, đĩa, cơ sở dữ liệu).
2	Sequential, Concurrent, Parallel	Phân biệt 3 trục: Tuần tự (lần lượt), Đồng thời (tiến triển xen kẽ trên 1 core), Song song (thực thi vật lý đồng thời trên đa nhân).
3	Synchronous và Asynchronous	Hợp đồng giao tiếp Caller-Callee: Synchronous bắt caller đợi tại chỗ; Asynchronous cho caller tiếp tục và trả kết quả sau.
4	Blocking và Non-blocking	Trạng thái luồng tại OS system call: Blocking đưa thread vào giấc ngủ (sleep); Non-blocking trả quyền ngay và dùng I/O multiplexing.
5	Vì sao hai cặp trên không đồng nghĩa	Chứng minh bằng ma trận 2x2: Có thể có Async + Blocking (anti-pattern) hoặc Sync + Non-blocking (polling).
6	Process, Thread, Coroutine	Nắm rõ cấu trúc bộ nhớ, chi phí tạo, chi phí context switch và phạm vi cô lập của 3 đơn vị thực thi.
7	I/O-bound, CPU-bound, Mixed workload	Nhận diện nút thắt qua tỷ lệ CPU Utilization và Wall time; áp dụng đúng công cụ cho từng loại.
8	Preemptive vs Cooperative scheduling	Preemptive do OS kernel ngắt cưỡng bức bằng timer; Cooperative do coroutine tự nguyện nhường quyền qua await.
9	Trạng thái vòng đời tác vụ (Task Lifecycle)	Hiểu 7 trạng thái: CREATED -> READY/PENDING -> RUNNING -> WAITING -> COMPLETED / FAILED / CANCELLED.
10	Vì sao Async không tự động là Parallel	Event loop chạy trên 1 luồng đơn duy nhất; tại mỗi thời điểm vì mô chỉ chạy 1 coroutine, không có đa nhân song song.

7.2. Năng lực Dự phòng (Backup) cho TV4: Event loop, Coroutine và Task

Để có thể phản biện và thuyết trình thay thế TV4 khi cần thiết, TV1 bắt buộc phải làm chủ cơ chế nội tại của asyncio (Năng lực 21–30):

- Cấu trúc Event loop: Gồm Ready Queue (hàng đợi FIFO chứa các Task sẵn sàng chạy), Timers Heap (cây min-heap quản lý thời gian chờ cho các lệnh sleep/timeout), và I/O Selector (bộ tích hợp epoll/kqueue/IOCP).
- Coroutine function vs Coroutine object: 'async def f():' chỉ là hàm sinh đối tượng; gọi 'f()' chỉ tạo ra generator/frame object ở trạng thái tạm dừng, chưa hề chạy cho đến khi được 'await' hoặc bọc vào 'asyncio.create_task()'.
- Bản chất asyncio.Task: Là một đối tượng bọc coroutine, kế thừa từ asyncio.Future, tự động đăng ký chính nó vào Event Loop để được lập lịch chạy độc lập.
- Structured Concurrency (Lập trình Đồng quy Có cấu trúc): Từ Python 3.11+, 'asyncio.TaskGroup' là chuẩn mực thay thế cho 'asyncio.gather()'. Đảm bảo mọi tác vụ con (child tasks) phải kết thúc hoặc bị hủy sạch sẽ trước khi thoát khỏi khối context manager 'async with', ngăn chặn triệt để tình trạng 'Task bị bỏ quên' (leaked / orphaned tasks).
- Xử lý lỗi nhóm (ExceptionGroup): Khi nhiều task con trong TaskGroup cùng quăng ngoại lệ, hệ thống gói tất cả vào một 'ExceptionGroup' và xử lý bằng cú pháp 'except*' đặc thù của Python 3.11+.

7.3. Hai Ví dụ Chuẩn & Hai Phản ví dụ (Anti-patterns) Kinh điển

✗ PHẢN VÍ DỤ 1 (LỖI CHÍ MẠNG): Phản ví dụ 1: Đưa lời gọi Blocking vào trong Coroutine làm tê liệt Event Loop

 Đoạn mã minh họa: Anti-pattern 1: Blocking call đóng băng toàn bộ Event loop

```
# SAI LẦM PHỔ BIẾN CỦA NGƯỜI MỚI:
import asyncio
import time

async def fetch_station_bad(station_id):
    print(f"Bắt đầu đọc trạm {station_id}")
    time.sleep(2) # NGUY HIỂM: time.sleep là BLOCKING! Nó làm đứng cả luồng OS!
    print(f"Xong trạm {station_id}")
    return 100

async def main_bad():
    # Dù dùng gather, 3 task này VẪN CHẠY TUẦN TỰ và mất 6 giây!
    # Trong suốt 6 giây này, toàn bộ event loop bị đóng băng 100%.
    await asyncio.gather(fetch_station_bad(1), fetch_station_bad(2), fetch_station_bad(3))
```

RCA: Phân tích nguyên nhân gốc rễ (Root Cause Analysis - RCA): 'time.sleep()' là một lời gọi hệ thống đồng bộ trực tiếp lên luồng OS. Nó không trả về đối tượng awaitable và không nhường quyền cho Event Loop. Do đó, Event loop không thể chuyển sang Task 2 hay Task 3, khiến chương trình mất đúng 6 giây tuần tự dù viết dưới mác async!

✔ **VÍ DỤ ĐÚNG 1 (CHUẨN COOPERATIVE ASYNC):** Ví dụ Đúng 1: Sử dụng Awaitable Non-blocking chuẩn xác

 Đoạn mã minh họa: Best Practice 1: Non-blocking await nhường quyền hợp tác

```
# CÁCH SỬA ĐÚNG:
import asyncio

async def fetch_station_good(station_id):
    print(f"Bắt đầu đọc trạm {station_id}")
    # asyncio.sleep đăng ký timer với event loop và NHƯỜNG QUYỀN (yield) ngay lập tức
    await asyncio.sleep(2)
```

```
print(f"Xong trạm {station_id}")
return 100

async def main_good():
    # 3 task chạy đồng thời, tổng thời gian CHỈ MẤT 2 GIÂY!
    await asyncio.gather(fetch_station_good(1), fetch_station_good(2), fetch_station_good(3))
```

✗ PHẢN VÍ DỤ 2 (LỖI TÀNG HÌNH): Phản ví dụ 2: Gọi Coroutine nhưng Quên Await (Never-awaited Coroutine)

 Đoạn mã minh họa: Anti-pattern 2: Coroutine bị gọi nhưng không bao giờ await

```
# LỖI TÀNG HÌNH THƯỜNG GẶP:
import asyncio

async def log_alert(msg):
    print(f"GHI CẢNH BÁO: {msg}")
    await asyncio.sleep(0.1)

async def process_data():
    # LỖI: Gọi hàm nhưng không có 'await' và không đưa vào Task!
    log_alert("Nhiệt độ vượt ngưỡng 50 độ!")
    # Cảnh báo KHÔNG BAO GIỜ ĐƯỢC GHI!
    # Python sẽ ném cảnh báo ngầm: RuntimeWarning: coroutine 'log_alert' was never awaited
```

RCA: Phân tích nguyên nhân gốc rễ (RCA): Trong Python, việc gọi 'log_alert()' chỉ đơn thuần khởi tạo một đối tượng coroutine nằm trên bộ nhớ RAM. Nếu không có 'await' hoặc không chuyển nó cho 'asyncio.create_task()', đối tượng này sẽ bị bộ thu gom rác (Garbage Collector) hủy bỏ khi hàm kết thúc mà mã bên trong chưa từng được chạy một dòng nào!

✓ VÍ DỤ ĐÚNG 2 (CÔ LẬP BLOCKING SANG LUỒNG PHỤ): Ví dụ Đúng 2: Tách biệt Blocking I/O bằng asyncio.to_thread

 Đoạn mã minh họa: Best Practice 2: Cô lập an toàn mã blocking bằng asyncio.to_thread

```
# CÁCH CÔ LẬP HÀM BLOCKING KHÔNG THỂ SỬA THÀNH ASYNC:
import asyncio
import time

def legacy_blocking_sensor_read(station_id):
    # Giả sử đây là thư viện C cũ của trạm cảm biến, bắt buộc phải blocking
    time.sleep(2)
    return {"station_id": station_id, "data": 42}

async def safe_worker(station_id):
    # Dùng asyncio.to_thread (Python 3.9+) đẩy sang ThreadPool riêng
    # Event loop hoàn toàn tự do, không bị nghẽn!
    data = await asyncio.to_thread(legacy_blocking_sensor_read, station_id)
    return data
```

8. Ngân hàng Câu hỏi Đánh giá & Đáp án Chi tiết của TV1

Để phục vụ buổi kiểm tra chéo (teach-back) và chuẩn bị bảo vệ trước Hội đồng chấm điểm, TV1 xây dựng bộ ngân hàng câu hỏi đồ sộ gồm: 12 câu trắc nghiệm (MCQ) có giải thích chi tiết đáp án đúng/sai,

6 câu hỏi tự luận có rubric chấm điểm 3 mức, 2 bài tập debug thực tế, và 10 câu hỏi phản biện bảo vệ miệng.

8.1. 12 Câu hỏi Trắc nghiệm (MCQ) Đa mức độ kèm Lời giải Thấu đáo

? Câu 1 (Mức độ: Nhận biết): Sự khác biệt căn bản nhất giữa Concurrency (Đồng thời) và Parallelism (Song song) là gì?

- A. Concurrency chỉ chạy trên nhiều máy tính; Parallelism chạy trên một máy tính.
- B. Concurrency là cấu trúc xử lý nhiều việc xen kẽ trong cùng khoảng thời gian; Parallelism là thực thi đồng thời vật lý tại cùng thời điểm trên nhiều lõi phần cứng.
- C. Concurrency luôn dùng đa luồng (multi-threading); Parallelism luôn dùng đa tiến trình (multi-processing).
- D. Concurrency chỉ dành cho CPU-bound; Parallelism chỉ dành cho I/O-bound.

👉 Đáp án đúng: B.

💡 *Giải thích:* Giải thích cận kề: Theo định nghĩa chuẩn của Rob Pike và khoa học máy tính, Concurrency là thuộc tính về cấu trúc (cùng tiến triển trong một khoảng thời gian Δt thông qua time-slicing), có thể đạt được trên 1 core duy nhất. Parallelism là thuộc tính về thực thi phần cứng (tại thời điểm t_0 chạy trên ≥ 2 core). Các phương án A, C, D đều sai về bản chất kỹ thuật.

? Câu 2 (Mức độ: Thông hiểu): Tại sao trên phiên bản CPython tiêu chuẩn, đa luồng (threading) KHÔNG làm tăng tốc độ xử lý của một vòng lặp tính toán toán học nặng (CPU-bound)?

- A. Vì hệ điều hành không hỗ trợ lập lịch cho các luồng của Python.
- B. Vì các luồng của Python không có ngăn xếp (stack) riêng.
- C. Do sự hiện diện của Global Interpreter Lock (GIL), tại mỗi thời điểm chỉ có đúng một luồng được phép thực thi bytecode Python.
- D. Vì Python tự động chuyển mã CPU-bound thành mã tuần tự mà không báo trước.

👉 Đáp án đúng: C.

💡 *Giải thích:* Giải thích cận kề: GIL là cơ chế khóa đơn để bảo vệ an toàn luồng cho bộ quản lý bộ nhớ đếm tham chiếu (reference counting) của CPython. Dù máy tính có bao nhiêu nhân, GIL ngăn cản các luồng chạy bytecode Python song song thực sự. Ngược lại, đa luồng còn làm chương trình chạy CHẬM HƠN do chi phí tranh chấp GIL và context switch liên tục.

? Câu 3 (Mức độ: Vận dụng): Điều gì xảy ra khi lập trình viên gọi lệnh 'time.sleep(5)' bên trong một hàm 'async def' đang chạy trên asyncio?

- A. Chỉ có coroutine đó bị tạm dừng 5 giây, các coroutine khác trong Event Loop vẫn tiếp tục chạy bình thường.
- B. Hệ điều hành tự động tạo một luồng mới để chạy đoạn ngủ 5 giây đó.
- C. Toàn bộ Event Loop thread bị chặn cứng trong 5 giây; tất cả các Task khác trên cùng loop đều bị đóng băng hoàn toàn.
- D. Python sẽ ném ra ngoại lệ BlockingIOError ngay lập tức khi biên dịch.

👉 Đáp án đúng: C.

💡 *Giải thích:* Giải thích cận kề: 'time.sleep()' là hàm blocking đồng bộ cấp hệ điều hành. Do asyncio hoạt động trên mô hình đơn luồng hợp tác (single-threaded cooperative loop), khi một coroutine gọi hàm blocking OS, luồng duy nhất của loop bị ngủ, không thể lấy các Task khác trong Ready Queue ra xử lý.

? Câu 4 (Mức độ: Thông hiểu): Bản chất của biểu thức 'await <expr>' trong Python là gì?

- A. Nó ép buộc CPU phải tạo ra một tiến trình mới chạy ngầm.

- B. Nó là một điểm tạm dừng (yield point) tự nguyện nhường quyền điều khiển về cho Event Loop nếu awaitable chưa sẵn sàng.
- C. Nó biến đổi mọi mã đồng bộ bên trong biểu thức trở thành mã non-blocking.
- D. Nó là câu lệnh khóa tài nguyên (Lock) ngăn các hàm khác truy cập biến dùng chung.

👉 **Đáp án đúng: B.**

💡 *Giải thích:* Giải thích cận kẽ: 'await' không làm mã nhanh hơn cũng không làm mã non-blocking. Nó kiểm tra xem awaitable (Future/Task) đã hoàn thành chưa; nếu chưa, nó tạm dừng frame hiện tại và trả quyền điều khiển về loop.

? **Câu 5 (Mức độ: Phân tích):** Theo Định luật Amdahl, nếu một chương trình có 25% thời lượng bắt buộc phải chạy tuần tự ($s = 0.25$), thì hệ số tăng tốc tối đa (Speedup) khi số lượng nhân CPU tiến tới vô cùng là bao nhiêu?

- A. Tăng tốc tối đa 25 lần.
- B. Tăng tốc tối đa 4 lần.
- C. Tăng tốc vô hạn (vô cùng lớn).
- D. Tăng tốc tối đa 75 lần.

👉 **Đáp án đúng: B.**

💡 *Giải thích:* Giải thích cận kẽ: Áp dụng công thức giới hạn tiệm cận của Amdahl: $S_{\max} = 1 / (1 - p) = 1 / s$. Với $s = 0.25$, $S_{\max} = 1 / 0.25 = 4.0$ lần. Dù có vô hạn core, phần tuần tự 25% luôn là bức tường chặn tốc độ tối đa ở mức 4 lần.

? **Câu 6 (Mức độ: Nhận biết):** Đơn vị thực thi nào sau đây có chi phí chuyển ngữ cảnh (Context Switch Overhead) nhỏ nhất?

- A. Tiến trình (Process).
- B. Luồng hệ điều hành (Kernel Thread).
- C. Hàm đồng quy (Coroutine trong asyncio).
- D. Cả ba đơn vị có chi phí chuyển ngữ cảnh như nhau.

👉 **Đáp án đúng: C.**

💡 *Giải thích:* Giải thích cận kẽ: Chuyển ngữ cảnh Process tốn ~5-15 μs (đổi Page Table, flush TLB); Thread tốn ~1-3 μs (lưu thanh ghi CPU tại kernel); Coroutine chỉ tốn ~20-50 ns ở user-space (chỉ đơn giản là đổi con trỏ frame trên heap của Python runtime, không qua kernel).

? **Câu 7 (Mức độ: Vận dụng):** Trong mô hình lập trình asyncio, khi nào thì hiện tượng Race Condition (Tranh chấp dữ liệu) VẪN CÓ THỂ xảy ra?

- A. Không bao giờ xảy ra vì asyncio chạy trên một luồng duy nhất.
- B. Xảy ra tại các điểm 'await' khi hai coroutine cùng đọc dữ liệu dùng chung, sau đó nhường quyền qua await, rồi quay lại ghi đè dữ liệu cũ.
- C. Chỉ xảy ra khi máy tính có từ 2 nhân CPU vật lý trở lên.
- D. Chỉ xảy ra khi dùng thư viện ngoài không phải chuẩn của Python.

👉 **Đáp án đúng: B.**

💡 *Giải thích:* Giải thích cận kẽ: Dù không có tranh đoạt cấp CPU chỉ thị, Race Condition mức ứng dụng (Application-level Race) vẫn xảy ra nếu chuỗi thao tác Read-Modify-Write bị ngắt quãng bởi một điểm 'await'. Đó là lý do asyncio vẫn phải cung cấp asyncio.Lock.

? **Câu 8 (Mức độ: Thông hiểu):** Trong bài toán thu thập dữ liệu từ 100 trạm cảm biến môi trường qua mạng Internet, nút thắt cổ chai chính (Bottleneck) thuộc loại nào?

- A. CPU-bound (Quá tải năng lực tính toán số học của CPU).
- B. Memory-bound (Tràn bộ nhớ RAM do kích thước dữ liệu cảm biến quá lớn).
- C. I/O-bound (Thời gian chờ phản hồi mạng và độ trễ gói tin chiếm ưu thế).
- D. Bus-bound (Nghẽn băng thông đường truyền bo mạch chủ).

🔑 Đáp án đúng: C.

💡 *Giải thích:* Giải thích cận kề: Thao tác gửi request và đợi phản hồi từ trạm cảm biến từ xa mất hàng chục đến hàng trăm mili-giây thời gian trễ mạng (Network latency), trong khi việc xử lý gói tin chỉ mất vài microsecond. CPU nhàn rỗi phần lớn thời gian -> I/O-bound.

? Câu 9 (Mức độ: Vận dụng): Hàm đo thời gian nào sau đây là CHUẨN XÁC NHẤT để đo lường khoảng thời gian thực thi (Elapsed duration) trong các bài benchmark Python?

- A. `time.time()`
- B. `time.perf_counter()`
- C. `datetime.datetime.now()`
- D. `time.process_time()`

🔑 Đáp án đúng: B.

💡 *Giải thích:* Giải thích cận kề: `'time.perf_counter()'` dùng đồng hồ phần cứng đơn điệu (monotonic clock) có độ phân giải cao nhất và không bị ảnh hưởng khi đồng hồ hệ thống bị điều chỉnh (NTP sync). `'time.time()'` có thể bị giật lùi; `'time.process_time()'` bỏ qua thời gian ngủ chờ I/O.

? Câu 10 (Mức độ: Thông hiểu): Tại sao trong CPython, 'ProcessPoolExecutor' thường được lựa chọn để giải quyết bài toán CPU-bound thay vì 'ThreadPoolExecutor'?

- A. Vì ProcessPoolExecutor sử dụng ít RAM hơn ThreadPoolExecutor.
- B. Vì mỗi Process sở hữu một Python Interpreter và GIL riêng biệt, cho phép thực thi song song thực sự trên nhiều nhân CPU.
- C. Vì ProcessPoolExecutor không cần tuần tự hóa dữ liệu qua pickle.
- D. Vì Process khởi động nhanh hơn Thread.

🔑 Đáp án đúng: B.

💡 *Giải thích:* Giải thích cận kề: Mỗi tiến trình con là một không gian địa chỉ độc lập với một GIL riêng, cho phép hệ điều hành phân bổ mỗi tiến trình lên một nhân CPU vật lý riêng biệt để chạy 100% công suất song song.

? Câu 11 (Mức độ: Phân tích): Chi phí ẩn (Overhead) nào sau đây KHÔNG ĐƯỢC tính đến trong công thức lý tưởng của Định luật Amdahl?

- A. Tỷ lệ phần trăm mã bắt buộc phải chạy tuần tự ($1 - p$).
- B. Số lượng nhân vi xử lý tham gia tính toán (N).
- C. Chi phí khởi tạo worker, chi phí tuần tự hóa (pickling), và độ trễ truyền thông liên tiến trình (IPC).
- D. Thời gian chạy ban đầu của chương trình trên 1 nhân $T(1)$.

🔑 Đáp án đúng: C.

💡 *Giải thích:* Giải thích cận kề: Định luật Amdahl giả định chi phí quản lý song song bằng 0. Trong thực tế, chi phí fork/spawn tiến trình, serialize dữ liệu qua pickle và nghẽn hàng đợi IPC có thể làm thời gian chạy thực tế lớn hơn rất nhiều so với lý thuyết.

? Câu 12 (Mức độ: Vận dụng): Lệnh nào sau đây là cách an toàn và đúng đắn nhất để bọc một hàm đọc cảm biến blocking trong mã asyncio mà không làm treo Event Loop?

- A. `await blocking_read_sensor()`

- B. `asyncio.create_task(blocking_read_sensor())`
- C. `await asyncio.to_thread(blocking_read_sensor)`
- D. `asyncio.run(blocking_read_sensor())`

👉 **Đáp án đúng:** C.

💡 **Giải thích:** Giải thích cận kẽ: 'asyncio.to_thread()' (từ Python 3.9+) đẩy hàm blocking sang một Thread riêng biệt trong ThreadPoolExecutor ngầm, đồng thời trả về một awaitable coroutine để Event Loop tiếp tục chạy việc khác mà không bị chặn.

8.2. 6 Câu hỏi Tự luận Chuyên sâu kèm Rubric Chấm điểm 3 Mức

📖 **Câu hỏi 1:** Phân tích tại sao một chương trình bất đồng bộ (asyncio) chạy trên 1 luồng đơn có thể phục vụ hàng chục nghìn kết nối mạng đồng thời, trong khi mô hình đa luồng truyền thống (multi-threading) lại gặp khó khăn?

💡 **Đáp án chuẩn:** Đáp án chuẩn: 1) Mô hình thread truyền thống gán mỗi kết nối cho 1 OS thread. Mỗi thread tốn từ 1-8 MB bộ nhớ ngăn xếp ảo; 10.000 thread sẽ ngốn từ 10-80 GB RAM gây sụp đổ hệ thống (C10K problem). 2) Chuyển ngữ cảnh giữa 10.000 thread khiến OS kernel mất toàn bộ thời gian cho context switch CPU (trashing). 3) Ngược lại, asyncio dùng 1 luồng duy nhất kết hợp cơ chế I/O multiplexing (epoll/IOCP). Mỗi kết nối chỉ là một coroutine tốn ~1-2 KB RAM. Chuyển ngữ cảnh diễn ra ở user-space tốn vài chục nanosecond, không tốn tài nguyên kernel.

📊 **Tiêu chí chấm điểm (Rubric):** Rubric: Kém (<50%): Chỉ nói async nhanh hơn không nêu được lý do bộ nhớ; Đạt (50-80%): Nêu được khác biệt bộ nhớ nhưng chưa giải thích được I/O multiplexing; Xuất sắc (90-100%): Phân tích đủ chi phí RAM stack, chi phí kernel context switch vs user-space context switch, và cơ chế epoll/IOCP.

📖 **Câu hỏi 2:** Giải thích nguyên tắc 'Correctness First, Performance Later' trong đo lường hiệu năng tính toán song song và vai trò của Sequential Baseline làm Correctness Oracle.

💡 **Đáp án chuẩn:** Đáp án chuẩn: 1) Trong lập trình đồng thời, việc tối ưu hóa hiệu năng rất dễ dẫn đến race condition, mất mát dữ liệu hoặc tính toán thiếu nhất quán. Một phiên bản chạy nhanh gấp 10 lần nhưng trả về kết quả sai lệch 5% là một thất bại kỹ thuật hoàn toàn. 2) Do đó, bản chuẩn tuần tự (Sequential Baseline) với tính chất đơn giản, không tranh chấp, chạy xác định được thiết lập làm 'Chân lý kiểm chứng' (Correctness Oracle). Mọi chế độ chạy đồng thời (Thread, Process, Async) bắt buộc phải chứng minh cho ra cùng một Result Digest (SHA-256) trước khi được phép bàn về tốc độ.

📊 **Tiêu chí chấm điểm (Rubric):** Rubric: Kém: Không hiểu thuật ngữ Correctness Oracle; Đạt: Nêu được ý nghĩa phải đúng trước khi nhanh; Xuất sắc: Giải thích trọn vẹn vai trò của Sequential Oracle, cơ chế kiểm chứng Result Digest và nguy cơ tối ưu sớm (premature optimization).

📖 **Câu hỏi 3:** Phân tích 4 loại chi phí Overhead thực tế khiến hệ số tăng tốc của hệ thống đa tiến trình (ProcessPool) bị suy giảm so với mô hình lý tưởng của Amdahl.

💡 **Đáp án chuẩn:** Đáp án chuẩn: 4 chi phí gồm: 1) Khởi tạo tiến trình (Process Startup): Hệ điều hành phải cấp phát bảng trang, khởi động lại Python runtime (spawn trên Windows mất 20-50ms). 2) Tuần tự hóa dữ liệu (IPC & Pickling): Dữ liệu gửi đến và nhận về từ tiến trình con phải qua pickle/unpickle, tốn CPU và độ trễ lớn nếu mảng dữ liệu nặng. 3) Tranh chấp tài nguyên & Đồng bộ: Tranh chấp hàng đợi công việc (work queue) và khóa đồng bộ. 4) Giới hạn phần cứng: Bão hòa băng thông bộ nhớ RAM (Memory Bus Saturation) và hiện tượng vô hiệu hóa bộ nhớ đệm (Cache Invalidation/TLB flush).

📊 **Tiêu chí chấm điểm (Rubric):** Rubric: Kém: Nêu dưới 2 loại chi phí; Đạt: Nêu được 3 chi phí cơ bản; Xuất sắc: Phân tích sâu sắc cả 4 chi phí, đặc biệt là pickle overhead trên Python và khác biệt nền tảng OS.

 **Câu hỏi 4:** Thế nào là Lập trình Đồng quy Có cấu trúc (Structured Concurrency) và tại sao 'asyncio.TaskGroup' trong Python 3.11+ lại ưu việt hơn 'asyncio.gather()'?

 **Đáp án chuẩn:** Đáp án chuẩn: 1) Structured Concurrency là mô hình bảo đảm vòng đời của các tác vụ con (child tasks) bị ràng buộc chặt chẽ trong phạm vi cấu trúc cú pháp của tác vụ cha. Không một tác vụ con nào được phép 'sống sót' hoặc 'bị bỏ rơi' (orphaned/leaked) khi tác vụ cha thoát ra. 2) Nhược điểm của gather: Nếu 1 task con bị lỗi, các task con khác vẫn tiếp tục chạy ngầm trong hư vô mà không bị hủy, gây rò rỉ tài nguyên. 3) TaskGroup khắc phục triệt để: Nếu 1 task con quăng lỗi, TaskGroup lập tức hủy (cancel) toàn bộ các task anh em còn lại, thu thập tất cả ngoại lệ vào một ExceptionGroup duy nhất, và bảo đảm dọn dẹp sạch sẽ tài nguyên.

 **Tiêu chí chấm điểm (Rubric):** Rubric: Kém: Không định nghĩa được structured concurrency; Đạt: Nêu được TaskGroup an toàn hơn nhưng chưa nêu rõ cơ chế hủy task anh em; Xuất sắc: Phân tích chuẩn xác cơ chế hủy lan truyền, ExceptionGroup và việc chống rò rỉ tài nguyên.

 **Câu hỏi 5:** Tại sao việc gọi coroutine mà quên từ khóa 'await' lại tạo ra một 'lỗi tàng hình' (invisible bug) trong Python và làm thế nào để phát hiện?

 **Đáp án chuẩn:** Đáp án chuẩn: 1) Trong Python, việc gọi 'f()' trên một coroutine function chỉ tạo ra một đối tượng coroutine nằm im trên bộ nhớ, nó không bao giờ tự chạy nếu không được await hoặc chuyển cho create_task. Đoạn mã bên trong không chạy nhưng chương trình cũng không ném ra exception làm sập app ngay lập tức, do đó trở thành lỗi tàng hình. 2) Khi hàm kết thúc, bộ thu gom rác xóa đối tượng và Python ném một cảnh báo ngầm: 'RuntimeWarning: coroutine was never awaited'. 3) Cách phát hiện: Bật chế độ Debug Mode của asyncio ('PYTHONASYNCIODEBUG=1' hoặc 'asyncio.run(main(), debug=True)') và bật cờ cảnh báo nghiêm ngặt của Python ('python -W error').

 **Tiêu chí chấm điểm (Rubric):** Rubric: Kém: Tưởng là chương trình tự động chạy; Đạt: Nêu được coroutine chưa được chạy và có RuntimeWarning; Xuất sắc: Nêu được cơ chế frame object, cách bật asyncio debug mode và cờ lọc cảnh báo hệ thống.

 **Câu hỏi 6:** Phân biệt sự khác nhau giữa việc sử dụng tính toán bất đồng bộ trên một máy tính cục bộ và một Hệ thống Phân tán (Distributed System) thực sự.

 **Đáp án chuẩn:** Đáp án chuẩn: 1) Về không gian bộ nhớ: Trên một máy tính (kể cả multi-process), hệ thống chia sẻ một phần cứng vật lý, đồng hồ hệ thống duy nhất (Single clock) và được giám sát bởi cùng một OS Kernel. Trong hệ thống phân tán, các nút máy tính nằm trên các máy chủ vật lý khác nhau qua mạng, không chia sẻ bộ nhớ, không có đồng hồ chung (No global clock). 2) Về mô hình lỗi (Fault model): Trên 1 máy, một lỗi crash là cục bộ hoặc sập cả máy. Trong hệ thống phân tán, xuất hiện các lỗi phức tạp: Phân mảnh mạng (Network Partition), thông điệp bị mất/trùng lặp, nút hoạt động chậm thất thường (straggler), đòi hỏi thuật toán đồng thuận phân tán (Raft, Paxos).

 **Tiêu chí chấm điểm (Rubric):** Rubric: Kém: Cho rằng hai mô hình là một; Đạt: Phân biệt được mạng và bộ nhớ; Xuất sắc: Phân tích thấu đáo về mô hình đồng hồ, sự phân mảnh mạng, ranh giới giữa local concurrency và distributed consensus.

8.3. 2 Bài tập Bắt lỗi & Phân tích Hệ thống Thực tế (Debugging Scenarios)

 **BÀI TẬP DEBUG 1:** Tình huống 1: Dịch vụ Thu thập Trạm Cảm biến bị Treo Cứng sau khi Bỏ sung Đoạn mã 'Làm ấm'

Mô tả sự cố: Một kỹ sư muốn thêm bước 'chờ khởi động cảm biến' trong hàm đọc trạm bất đồng bộ. Sau khi triển khai đoạn mã dưới đây, hệ thống mất tới 40 giây để đọc 8 trạm (thay vì 0.5 giây như trước), và giao diện giám sát báo 'Connection Timed Out'. Hãy chẩn đoán nguyên nhân và sửa lại cho đúng.

 **Đoạn mã minh họa:** Mã nguồn gây sự cố treo hệ thống

```
# MÃ NGUỒN CÓ LỖI (BUGGY CODE):
import asyncio
import time

async def fetch_sensor_data(station_id):
    print(f"Đang kết nối trạm {station_id}...")
    # Bước khởi động mô phỏng:
    time.sleep(5) # <-- ĐOẠN MÃ GÂY LỖI
    return {"station_id": station_id, "temp": 28.5}

async def run_monitoring_hub():
    stations = [f"STA-{i:03d}" for i in range(1, 9)]
    tasks = [fetch_sensor_data(sid) for sid in stations]
    results = await asyncio.gather(*tasks)
    return results
```

Chẩn đoán & Khắc phục: Chẩn đoán lỗi: Việc sử dụng 'time.sleep(5)' bên trong coroutine làm chặn đứng toàn bộ luồng duy nhất của Event Loop. Mặc dù sử dụng 'asyncio.gather', cả 8 coroutine không thể chạy đồng thời mà bị ép chạy tuần tự nối tiếp nhau: 8 trạm x 5 giây = 40 giây! Trong suốt 40 giây này, không một sự kiện I/O nào khác được xử lý.

Giải pháp khắc phục: Thay thế 'time.sleep(5)' bằng 'await asyncio.sleep(5)'. Lệnh này đăng ký một timer với Event Loop và ngay lập tức nhường quyền điều khiển. Khi đó, cả 8 trạm cùng đếm lùi 5 giây song song, và toàn bộ quá trình hoàn tất chỉ sau đúng 5 giây!

🔧 BÀI TẬP PHÂN TÍCH 2: Tình huống 2: Phân tích Định luật Amdahl cho Hệ thống Xử lý Ảnh Vệ tinh Môi trường

Mô tả bài toán: Trung tâm giám sát môi trường cần chạy thuật toán phân tích chỉ số ô nhiễm không khí từ ảnh vệ tinh. Hệ thống đo đạc cho thấy: Công đoạn tải ảnh và giải nén ban đầu chiếm 30% tổng thời gian (không thể song song hóa vì tuần tự tệp tin). Công đoạn tính toán ma trận pixel chiếm 70% thời gian (có thể chia nhỏ song song hoàn toàn).

Yêu cầu:

1. Tính Hệ số Tăng tốc lý thuyết (Speedup) khi chạy trên máy chủ 4 nhân và 16 nhân.
2. Ban giám đốc dự định đầu tư cụm máy chủ 128 nhân CPU với kỳ vọng tăng tốc độ xử lý lên 50 lần. Dưới góc độ kỹ sư PDC, bạn hãy dùng Định luật Amdahl để tư vấn xem kỳ vọng này có khả thi không và giải thích tại sao.

Lời giải & Tư vấn Kỹ thuật: Lời giải chi tiết:

1. Tỷ lệ phần tuần tự $s = 0.30$; Tỷ lệ phần song song $p = 0.70$.

- Với máy chủ 4 nhân ($N = 4$): $S(4) = 1 / [0.30 + 0.70/4] = 1 / [0.30 + 0.175] = 1 / 0.475 = 2.105$ lần.

- Với máy chủ 16 nhân ($N = 16$): $S(16) = 1 / [0.30 + 0.70/16] = 1 / [0.30 + 0.04375] = 1 / 0.34375 = 2.909$ lần.

2. Tư vấn cho Ban giám đốc:

- Tính toán giới hạn tiệm cận cực đại: $S_{\text{max}} = \lim (N \rightarrow \infty) S(N) = 1 / (1 - p) = 1 / 0.30 = 3.333$ LẦN.

- **KẾT LUẬN:** Kỳ vọng tăng tốc 50 lần là **HOÀN TOÀN BẤT KHẢ THI VỀ MẶT TOÁN HỌC!** Dù có trang bị 128 nhân, 1.000 nhân hay 1 triệu nhân, tốc độ tối đa không bao giờ vượt quá 3.33 lần do phần tuần tự 30% chi phối. Để đạt được mục tiêu tăng tốc cao hơn, giải pháp không phải là mua thêm CPU, mà là phải tái cấu trúc thuật toán để giảm tỷ lệ phần tuần tự (ví dụ: dùng định dạng ảnh cho phép đọc streaming song song).

8.4. Bộ 10 Câu hỏi Phản biện Bảo vệ Miệng (Oral Defense) với Giảng viên

🎧 1. Giảng viên hỏi: 'Tại sao em khẳng định Concurrency khác hoàn toàn Parallelism? Cho một ví dụ cụ thể trong đời sống?'

🗨️ TV1 trả lời: Concurrency là về cấu trúc sắp đặt; Parallelism là về thực thi vật lý tại một thời điểm. Ví dụ đời thường: Một nhân viên pha cà phê vừa nhận đơn, vừa bấm nút máy pha, vừa đưa bánh cho khách — đó là Concurrency (1 người xử lý nhiều việc xen kẽ). Còn hai nhân viên pha chế cùng lúc đứng pha hai tách cà phê trên hai máy khác nhau — đó là Parallelism (đa nhân vật lý).

🎧 2. Giảng viên hỏi: 'Một chương trình chạy trên 1 luồng duy nhất có thể đạt được Concurrency không?'

🗨️ TV1 trả lời: Hoàn toàn có thể. Đó chính là mô hình của NodeJS và Python asyncio. Event Loop trên 1 luồng đơn có thể điều phối hàng nghìn Task bằng cơ chế xen kẽ khi các Task chờ I/O.

🎧 3. Giảng viên hỏi: 'API bất đồng bộ (async API) có luôn luôn là Non-blocking không?'

🗨️ TV1 trả lời: Không. Đây là hai khái niệm độc lập. Một hàm khai báo 'async def' nhưng bên trong gọi 'time.sleep()' hoặc 'requests.get()' thì nó là một Async API nhưng lại mang bản chất Blocking. Nó tạo ra lỗi làm treo Event Loop.

🎧 4. Giảng viên hỏi: 'Từ khóa await có làm dừng (block) luồng của hệ điều hành không?'

🗨️ TV1 trả lời: Không. 'await' chỉ tạm dừng (suspend) coroutine hiện tại và nhường quyền điều khiển về cho Event Loop. Luồng OS vẫn tiếp tục chạy để phục vụ các Task khác trong Ready Queue.

🎧 5. Giảng viên hỏi: 'Tại sao bài toán I/O-bound và CPU-bound lại cần hai chiến lược xử lý hoàn toàn khác nhau?'

🗨️ TV1 trả lời: Vì nút thắt cổ chai khác nhau. I/O-bound nghẽn vì chờ đợi ngoại vi -> giải pháp là chồng lấp thời gian chờ (dùng asyncio hoặc ThreadPool). CPU-bound nghẽn vì chu kỳ tính toán của chip -> giải pháp là chia nhỏ việc lên nhiều core vật lý (dùng ProcessPool để vượt qua GIL của Python).

🎧 6. Giảng viên hỏi: 'Tại sao việc tăng thêm số lượng worker trong pool đôi khi lại làm chương trình chạy chậm hơn?'

🗨️ TV1 trả lời: Do chi phí Overhead và hiện tượng Oversubscription. Khi số worker vượt quá số nhân vật lý hoặc ngưỡng nghẽn I/O, hệ điều hành mất nhiều thời gian cho việc Context Switching (trashing), tranh chấp khóa bộ nhớ và bão hòa hàng đợi.

🎧 7. Giảng viên hỏi: 'Latency và Throughput có thể cho ra hai kết luận trái ngược nhau về một hệ thống không?'

🗨️ TV1 trả lời: Có. Ví dụ, một hệ thống xử lý theo lô (batching) có thể tăng Throughput từ 100 lên 1.000 request/giây, nhưng Latency của một request đơn lẻ lại tăng từ 10ms lên 50ms do phải đợi gom đủ lô.

🎧 8. Giảng viên hỏi: 'Định luật Amdahl bỏ qua những loại chi phí thực tế nào?'

🗨️ TV1 trả lời: Amdahl giả định chi phí quản lý song song bằng 0. Trong thực tế, định luật bỏ qua: chi phí khởi tạo process/thread, chi phí tuần tự hóa pickle, chi phí IPC qua pipe/socket, và chi phí tranh chấp khóa đồng bộ.

🎧 9. Giảng viên hỏi: 'Tại sao nhóm phải kiểm tra cùng kết quả (Result Digest) trước khi đo tốc độ?'

🗨️ TV1 trả lời: Tuân thủ nguyên tắc 'Correctness First'. Một phiên bản chạy nhanh gấp 10 lần nhưng bỏ sót dữ liệu hoặc tính sai kết quả là vô giá trị. Digest bảo đảm mọi chế độ đều giải quyết cùng một bài toán với độ chính xác tương đương.

🎤 10. Giảng viên hỏi: 'Lập trình bất đồng bộ trên một máy tính có phải là Tính toán Phân tán không?'

💬 TV1 trả lời: Không. Tính toán phân tán đòi hỏi các nút máy tính nằm trên các máy chủ vật lý độc lập, giao tiếp qua mạng, không có bộ nhớ chung và không có đồng hồ chung. Lập trình async trên một máy vẫn nằm trọn trong một hệ điều hành và một máy vật lý duy nhất.

9. Kịch bản Thuyết trình (Speaker Notes) & Đặc tả Slide

Để đảm bảo bài thuyết trình của TV1 chuẩn xác đến từng giây, hấp dẫn và giữ vững mạch logic của nhóm, TV1 xây dựng 3 kịch bản theo các khung thời lượng: Kịch bản Thang máy 1 phút, Kịch bản Rút gọn 3 phút, và Kịch bản Chuẩn mực 8 phút tương ứng với 8 slide chuẩn.

9.1. Kịch bản Thuyết trình 1 phút, 3 phút và 8 phút

🕒 **KỊCH BẢN 1 PHÚT (ELEVATOR PITCH):** 'Kính thưa Thầy Cô và các bạn! Trong thế giới phần mềm, phần lớn thời gian hệ thống bị lãng phí không phải vì CPU yếu, mà vì CPU phải ngồi chờ dữ liệu mạng và ổ đĩa một cách vô ích. Phần trình bày của em thiết lập nền tảng tư duy cốt lõi của Chương 4: phân biệt chính xác giữa đồng thời và song song, giải mã ma trận 2x2 giữa bất đồng bộ và non-blocking, và chỉ ra lý do tại sao thêm nhiều worker chưa chắc đã chạy nhanh hơn thông qua Định luật Amdahl. Đồng thời, em cung cấp bản chuẩn tuần tự làm thước đo tính đúng đắn cho toàn bộ hệ thống demo của nhóm. Xin mời các bạn cùng bước vào chi tiết!'

🕒 **KỊCH BẢN 3 PHÚT (BẢN RÚT GỌN CHO 6 SLIDE):** 'Kính thưa Thầy Cô! Em phụ trách Khối 1: Nền tảng và Bản đồ tư duy.'

Điểm mấu chốt đầu tiên: Concurrency là cấu trúc sắp đặt nhiều việc xen kẽ để tận dụng thời gian rảnh; còn Parallelism là chạy song song vật lý trên đa nhân. Nhiều người lầm tưởng async luôn non-blocking, nhưng ma trận 2x2 cho thấy nếu gọi hàm blocking trong async, toàn bộ hệ thống sẽ bị đóng băng!

Điểm mấu chốt thứ hai: Đơn vị thực thi gồm Process (cô lập, tốn RAM), Thread (chung nhớ, bị kim hãm bởi GIL CPython), và Coroutine (siêu nhẹ, lập lịch hợp tác trong 1 luồng duy nhất). Từ đó, ta phân loại: I/O-bound dùng asyncio/thread; CPU-bound bắt buộc dùng process.

Điểm mấu chốt thứ ba: Tăng tốc luôn có trần giới hạn bởi Định luật Amdahl do phần tuần tự bắt buộc và chi phí overhead thực tế. Nhóm em cam kết nguyên tắc 'Đúng trước - Nhanh sau' với bản chuẩn Sequential Baseline làm mốc đối soát. Ngay sau đây, bạn TV2 sẽ trình bày cách giao việc cho ThreadPool!'

9.2. Kịch bản Chi tiết 8 phút cho 8 Slide Chuẩn (Slide 1 đến 8)

Slide 1 (0:00 - 1:00) — Bài toán và Câu hỏi Trung tâm

Lời giảng: 'Kính thưa Thầy Cô! Hãy tưởng tượng chúng ta quản lý một trung tâm quan trắc môi trường với hàng trăm trạm cảm biến. Mỗi trạm mất 0.2 giây để gửi dữ liệu qua mạng. Nếu đọc tuần tự 8 trạm, CPU mất gần 2 giây chỉ để ngồi chờ nhàn rỗi! Ta đặt ra 3 câu hỏi kỹ thuật: 1) Có thể chồng lấp các khoảng chờ I/O không? 2) Phần tính toán có thể chạy song song không? 3) Chi phí overhead là bao nhiêu? Để trả lời, ta phải phân biệt rõ các khái niệm ở slide tiếp theo.'

Câu hỏi chuyển: Nhiều việc cùng tiến triển có nhất thiết phải chạy cùng một thời điểm không?

Slide 2 (1:00 - 2:00) — Tuần tự, Đồng thời và Song song

Lời giảng: 'Quan sát 3 trục thời gian trên slide: Xử lý tuần tự chạy lần lượt từng việc A rồi đến B. Xử lý đồng thời (Concurrency) trên 1 core chia nhỏ việc và xen kẽ nhau — khi A chờ mạng, CPU tranh thủ tính việc B. Xử lý song song (Parallelism) bắt buộc phải có từ 2 core vật lý trở lên, tại cùng một tích tắc chạy

cả A và B độc lập. Như Rob Pike đã nói: Đồng thời là cấu trúc chương trình; Song song là thuộc tính phần cứng thực thi!

Câu hỏi chuyển: Vậy Đồng bộ và Bất đồng bộ nằm ở trục nào so với Blocking và Non-blocking?

Slide 3 (2:00 - 3:00) — Đồng bộ, Bất đồng bộ, Blocking và Non-blocking

Lời giảng: 'Đây là cái bẫy lớn nhất! Đồng bộ/Bất đồng bộ là hợp đồng giữa Bên gọi và Bên được gọi: Caller chờ lấy kết quả ngay hay nhận handle rồi đi làm việc khác. Còn Blocking/Non-blocking là trạng thái luồng ở hệ điều hành: luồng có bị ngủ không. Ma trận 2x2 chỉ ra: Nguy hiểm nhất là ô số 3 — Bất đồng bộ nhưng Blocking! Khi bạn viết `async def` mà bên trong gọi `time.sleep`, bạn đang đóng băng cả Event loop!'

Câu hỏi chuyển: Ai là người thực thi công việc bên dưới và cơ chế lập lịch ra sao?

Slide 4 (3:00 - 4:00) — Process, Thread, Coroutine và Scheduler

Lời giảng: 'Bảng so sánh trên slide mở xẻ 3 đơn vị: Process có vùng nhớ riêng, chịu lỗi tốt nhưng tốn hàng chục MB và chi phí fork/IPC rất nặng. Thread dùng chung bộ nhớ, nhưng trên CPython bị khóa GIL ngăn không cho chạy đa nhân thực sự. Coroutine là giải pháp đột phá: chỉ tốn 1-2 KB, chạy trong user-space, đổi ngữ cảnh mất vài chục nanosecond theo cơ chế lập lịch hợp tác (cooperative) qua `await`.'

Câu hỏi chuyển: Ta chọn đơn vị nào cho bài toán chờ mạng và bài toán tính toán?

Slide 5 (4:00 - 5:00) — I/O-bound, CPU-bound và Mixed Workload

Lời giảng: 'Quy tắc bất biến: Phân loại nút thắt trước khi chọn công nghệ! Nếu CPU utilization dưới 10%, thời gian mất vào mạng -> Đó là I/O-bound, hãy chọn `asyncio` hoặc `ThreadPool`. Nếu CPU utilization 100% -> Đó là CPU-bound, trên Python hãy chọn `ProcessPool`. Với bài toán trạm cảm biến hỗn hợp của nhóm: Tầng nhận dữ liệu dùng Async, tầng tính toán dùng `ProcessPool`!'

Câu hỏi chuyển: Làm sao ta biết giải pháp mới thật sự chạy tốt hơn bản cũ?

Slide 6 (5:00 - 6:00) — Đo đúng trước khi kết luận

Lời giảng: 'Một chương trình chạy nhanh nhưng trả về kết quả sai là vô giá trị! Nhóm em đặt nguyên tắc: Correctness First! Phải dùng `time.perf_counter`, có warm-up làm ấm cache, chạy lặp lại 10 lần và lấy giá trị Trung vị (Median), tuyệt đối không in console trong vùng đo. Và điều kiện tiên quyết: Mã băm Result Digest của bản song song phải trùng 100% với bản chuẩn tuần tự!'

Câu hỏi chuyển: Liệu tăng số lượng worker lên vô hạn có làm tốc độ tăng vô hạn không?

Slide 7 (6:00 - 7:00) — Giới hạn Tăng tốc & Định luật Amdahl

Lời giảng: 'Định luật Amdahl chỉ ra một sự thật cay đắng: Trần tốc độ bị chặn bởi phần tuần tự bắt buộc! Nếu hệ thống có 20% công việc phải chạy tuần tự (khởi tạo, tổng hợp), thì dù có 1 triệu core, tốc độ tối đa không bao giờ vượt quá 5 lần! Đó là chưa kể 4 chi phí overhead thực tế: khởi tạo process, serialize pickle, tranh chấp lock. Nhiều worker quá sẽ gây nghẽn và làm chậm máy!'

Câu hỏi chuyển: Vậy nhóm em đã thiết lập chuẩn mực thực nghiệm đầu vào ra sao?

Slide 8 (7:00 - 8:00) — Baseline Demo và Bản đồ Chương 4

Lời giảng: 'Để làm mốc quy chiếu khoa học cho cả 5 chế độ thực thi tiếp theo, em đã hoàn thành module `sensor_baseline.py` với Data Contract chuẩn và Sequential Oracle. Kết quả chạy tuần tự trên 8 trạm mất 1.15 giây, digest SHA-256 là mốc bất biến. Bản đồ Chương 4 đã sẵn sàng: Xin nhường lại sân khấu cho bạn TV2 giải thích cách giao việc cho `ThreadPoolExecutor`!'

10. Biên bản Review Chéo TV1 ↔ TV4, Quản trị Rủi ro & Tài liệu Tham khảo

10.1. Biên bản và Quy trình Review Chéo giữa TV1 và TV4

Cặp bài trùng TV1 (Nền tảng khái niệm) và TV4 (Event loop, Coroutine và Task) có mối liên hệ mật thiết nhất trong dự án. Hai thành viên thực hiện cơ chế Review chéo bắt buộc theo 5 tiêu chí định lượng:

Tiêu chí Review	Nội dung Kiểm tra của TV1 đối với TV4	Nội dung Kiểm tra của TV4 đối với TV1	Trạng thái
1. Thuật ngữ (Terminology)	Kiểm tra TV4 không dùng lẫn lộn giữa Task và Coroutine, không gọi awaitable là callback	Kiểm tra TV1 phân biệt chuẩn xác giữa asynchrony và non-blocking trong ma trận 2x2	ĐẠT (100%)
2. Tính Đúng đắn Mã nguồn	Rà soát mã của TV4 không có lời gọi blocking (time.sleep, requests) bên trong coroutine	Rà soát module sensor_baseline của TV1 chạy đúng logic và pass đủ 6 test case	ĐẠT (100%)
3. Trực quan hóa (Diagrams)	Kiểm tra sơ đồ Event loop của TV4 có vẽ rõ Ready Queue, Timers Heap và I/O Selector	Kiểm tra sơ đồ Timeline của TV1 thể hiện rõ trạng thái CPU wait và idle	ĐẠT (100%)
4. Giới hạn & Ngoại lệ	Kiểm tra TV4 có xử lý kịch bản hủy tác vụ (cancellation) và ExceptionGroup trong TaskGroup	Kiểm tra TV1 có phân tích đầy đủ các overhead thực tế ngoài định luật Amdahl	ĐẠT (100%)
5. Năng lực Thuyết trình thay	TV1 diễn tập thuyết trình thành công slide 25–32 của TV4 trong thời gian 8 phút	TV4 diễn tập thuyết trình thành công slide 1–8 của TV1 trong thời gian 8 phút	ĐẠT (100%)

10.2. Bảng Quản trị Rủi ro Kỹ thuật của TV1 (Risk Register)

Mã Rủi ro	Mô tả Rủi ro Tiềm ẩn	Mức độ	Kế hoạch Ứng phó & Biện pháp Giảm thiểu
RISK-01	Người nghe nhầm lẫn giữa Concurrency và Parallelism	Cao	Bắt buộc chiếu sơ đồ 3 timeline so sánh 4 task A-D; nhấn mạnh câu nói kinh điển của Rob Pike.
RISK-02	Nội dung lý thuyết quá rộng làm cháy giáo án 8 phút	Trung bình	Khóa cứng kịch bản nói 8 slide chuẩn; đưa toàn bộ chứng minh toán học và bảng API chi tiết vào phụ lục P01-P03.
RISK-03	Mã demo baseline bị sai lệch kết quả giữa các lần chạy	Cao	Cố định seed=42 cho random generator; tính toán mã băm Result Digest (SHA-256) để kiểm chứng tính bất biến.
RISK-04	TV4 gặp sự cố vắng mặt trong buổi báo cáo chính thức	Trung bình	TV1 đóng vai trò Backup chính thức, đã hoàn thành

			diễn tập và sẵn sàng thuyết trình thay thế toàn bộ slide 25-32.
--	--	--	---

10.3. Danh mục Tài liệu Tham khảo Học thuật & Căn cứ Kỹ thuật

- **1. Python Official Documentation (Python 3.14.7, 2026):** 'asyncio — Asynchronous I/O', 'concurrent.futures — Launching parallel tasks', 'threading — Thread-based parallelism', 'multiprocessing — Process-based parallelism'. Nguồn trực tuyến: <https://docs.python.org/3.14/>
- **2. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018):** 'Operating System Concepts' (10th Edition). John Wiley & Sons. (Nền tảng về Process, Thread, Preemptive Scheduling và Memory Management).
- **3. Amdahl, G. M. (1967):** 'Validity of the single processor approach to achieving large scale computing capabilities'. AFIPS Conference Proceedings, Vol. 30, pp. 483–485. (Bài báo gốc xác lập Định luật Amdahl).
- **4. Pike, R. (2012):** 'Concurrency is not Parallelism'. Google Tech Talk. (Nguồn gốc phân biệt bản chất Concurrency vs Parallelism).
- **5. Zacccone, G. (2015):** 'Python Parallel Programming Cookbook'. Packt Publishing Ltd. (Giáo trình tham khảo chính thức của môn học PDC tại CTUET).
- **6. Pierfederici, F. (2016):** 'Distributed Computing with Python'. Packt Publishing Ltd. (Giáo trình tham khảo phân biệt Asynchronous cục bộ và Phân tán).
- **7. Fowler, M. (2014):** 'Memory Models and Concurrency Primitives'. Martinowler.com. (Phân tích race condition và memory safety).

 **KẾT LUẬN VÀ BÀN GIAO HỒ SƠ CHUYÊN MÔN TV1:** Hồ sơ của TV1 hoàn tất với đầy đủ 8 Work Packages (100 điểm tải, 48 giờ lao động kỹ thuật). Tất cả các tài sản số (Word Report, Slide PowerPoint, Source Code, Test Suite) đều đồng bộ và sẵn sàng phục vụ buổi báo cáo chuyên môn!